

TFM: SLM with Transformers and GRU

Guillem Gonzalo Silveira

April 2026

c. 4186 words

Abstract

This thesis investigates the design and implementation of a Small Language Model (SLM) using a hybrid architecture that integrates Transformer *multi-head attention* mechanisms with Gated Recurrent Units (GRU). Rather than relying on purely attention-based designs, the proposed approach combines global semantic understanding through attention with efficient sequential processing via recurrence, reducing memory requirements during both training and inference. The model is trained from scratch on diverse open-source conversational and instruction-following datasets, followed by comprehensive evaluation against established baselines using standard language modeling metrics. The work includes full production infrastructure validation through cloud-based deployments, automated testing pipelines, and cost-efficiency analysis. Results demonstrate that hybrid architectures can achieve competitive performance with reduced computational footprint, providing a viable alternative for developing specialized language models in resource-constrained environments.

Abstract

Este trabajo investiga el diseño e implementación de un Small Language Model (SLM) utilizando una arquitectura híbrida que integra mecanismos de atención Transformer *multi-head attention* con Gated Recurrent Units (GRU). En lugar de depender exclusivamente de arquitecturas basadas en atención, el enfoque propuesto combina la comprensión semántica global mediante atención con el procesamiento secuencial eficiente mediante recurrencia, reduciendo los requisitos de memoria durante entrenamiento e inferencia. El modelo se entrena desde cero sobre datasets públicos diversos de diálogos e instrucciones, seguido de evaluación exhaustiva contra líneas base establecidas utilizando métricas estándar de modelado del lenguaje. El trabajo incluye validación completa de infraestructura en producción mediante despliegues en la nube, pipelines de testing automatizados, y análisis de eficiencia económica. Los resultados demuestran que arquitecturas híbridas pueden alcanzar rendimiento competitivo con menor costo computacional, proporcionando una alternativa viable para desarrollar modelos de lenguaje especializados en entornos con recursos limitados.

Abstract

Aquest treball investiga el disseny i implementació d'un Small Language Model (SLM) utilitzant una arquitectura híbrida que integra mecanismes d'atenció Transformer *multi-head attention* amb Gated Recurrent Units (GRU). En lloc de dependre exclusivament d'arquitectures basades en atenció, l'enfocament proposat combina la comprensió semàntica global mitjançant atenció amb el processament seqüencial eficient mitjançant recurrència, reduint els requisits de memòria durant entrenament i inferència. El model s'entrena des de zero sobre datasets públics diversos de diàlegs i instruccions, seguit d'avaluació exhaustiva contra línies base establertes utilitzant mètriques estàndard de modelatge del llenguatge. El treball inclou validació completa d'infraestructura en producció mitjançant desplegaments al núvol, pipelines de testing automatitzats, i anàlisi d'eficiència econòmica. Els resultats demostren que arquitectures híbrides poden assolir rendiment competitiu amb menor cost computacional, proporcionant una alternativa viable per desenvolupar models de llenguatge especialitzats en entorns amb recursos limitats.

Contents

1	Introducción	7
1.1	Contexto y Motivación	7
1.2	Problema y Propuesta	7
1.3	Objetivos	7
1.4	Contribuciones	8
1.5	Estructura del Documento	8
2	Estado del Arte	10
2.1	Evolución de los Modelos de Lenguaje: De LLM a SLM	10
2.2	Limitaciones de los Transformers Puros	10
2.3	Fundamentos: Redes Neuronales Recurrentes (RNN)	10
2.4	LSTM y GRU: Soluciones a Dependencias Largo Plazo	11
2.5	El Mecanismo de Atención y el Nacimiento de los Transformers	11
2.6	Arquitecturas Híbridas y Recurrencia Moderna	12
3	Fundamentos Teóricos	14
3.1	El Mecanismo de Atención	14
3.2	Multi-Head Attention	14
3.3	Gated Recurrent Units (GRU)	14
3.3.1	Justificación de GRU frente a LSTM	15
3.4	La Hipótesis Híbrida: Análisis de Complejidad	15
4	Datos y Datasets para Pre-entrenamiento	17
4.1	Principios de Diseño para la Mezcla de Datos	17
4.2	Descripción Detallada de Datasets	17
4.2.1	OpenAssistant (OASST1)	17
4.2.2	ShareGPT	18
4.2.3	Alpaca	18
4.2.4	UltraChat	19
4.2.5	The Stack (YAML) - Gated, No Incluido	20
4.3	Estrategia de Mezcla: Ponderación y Proporción	20
4.3.1	Justificación de Pesos	20
4.4	Procesamiento de Datos y Formato Apache Arrow	21
4.4.1	Pipeline de Descargar y Procesamiento	21
4.4.2	Ventajas del Formato Apache Arrow	22

4.5	Estadísticas Finales de Entrenamiento	22
4.6	Validación de Cobertura y Diversidad	23
5	Mejoras Arquitectónicas e Implementación del Modelo Híbrido	24
5.1	Arquitectura Original y sus Limitaciones	24
5.2	Mejora 1: Persistent GRU Hidden States (CRÍTICA)	24
5.2.1	Implementación Técnica	24
5.2.2	Impacto en la Memoria Recurrente	25
5.3	Mejora 2: Reorden de Componentes (GRU Pre-Attention)	25
5.4	Mejora 3: Inicialización Ortogonal para Componentes RNN	26
5.5	Mejora 4: Gradient Clipping (CRÍTICA)	26
5.6	Mejora 5: Logging de Contribuciones de Componentes	27
5.7	Mejora 6: Validation Loop	27
5.8	Mejora 7: Herramienta de Análisis Arquitectónico	28
5.9	Mejora 8: Suite de Tests Automatizados	28
5.10	Impacto Cuantificable en la Convergencia	29
5.10.1	Velocidad de Convergencia	29
5.10.2	Estabilidad de Gradientes	29
5.10.3	Visibilidad y Debuggabilidad	29
5.11	Configuración Recomendada para Entrenamiento	29
6	Diseño e Implementación de la Arquitectura Híbrida	31
6.1	Estrategia de Curación y Mezcla de Datos (Data Mixing)	31
6.1.1	Selección de Datasets de Propósito General	31
6.1.2	Infraestructura de Datos: Formato Apache Arrow	31
6.1.3	Inyección de Conocimiento de Dominio (Fase Final)	32
6.1.4	Distribución del Dataset de Pre-entrenamiento	32
6.2	Arquitectura del Modelo: Hybrid Transformer-GRU	32
6.2.1	El Bloque Híbrido (HybridBlock)	32
6.2.2	Optimización del Cómputo de Atención: Flash Attention	33
6.2.3	Especificaciones Técnicas y Parámetros	33
6.3	Optimización y Estabilidad del Entrenamiento	34
6.4	Expectativas de Convergencia y Métricas de Éxito	34
7	Evaluación y Benchmarking	36
7.1	Entorno y Configuración del Benchmark	36
7.2	Métricas de Evaluación	36
7.2.1	Calidad del Modelo	36
7.2.2	Eficiencia Computacional	37
7.2.3	Huella de Recursos	37
7.3	Análisis de Estabilidad	38
7.4	Resumen de Resultados	38
8	Infraestructura Cloud y Análisis de Costes	39
8.1	Ecosistema AWS: Provisión de Recursos	39
8.1.1	Repositorio de Imágenes: Amazon ECR	39
8.2	Selección del Hardware: Familia G6 y NVIDIA L4	39

8.2.1	Comparativa de Familias de Aceleración	39
8.2.2	Arquitectura NVIDIA Ada Lovelace en G6	40
8.3	Gestión de Infraestructura mediante Terraform	40
8.3.1	Automatización y Prerrequisitos	40
8.4	Optimización de Memoria: Batching y Aprovechamiento de VRAM	41
8.4.1	Estrategia de Gradient Accumulation	41
8.5	Evaluación de Rendimiento vs Coste	42
8.6	Mejoras Iterativas de la Arquitectura durante el Entrenamiento	42
8.6.1	Persistent GRU Hidden States	42
8.6.2	Posicionamiento Pre-Attention del GRU	42
8.6.3	Inicialización Ortogonal	42
8.6.4	Gradient Clipping	42
8.6.5	Component Contribution Logging	43
8.6.6	Validation Loop	43
8.7	Evolución del Hardware: Migración a G7e	43
9	MLOps: Despliegue y Puesta en Producción	44
9.1	Arquitectura de Servicios e Idempotencia	44
9.2	Reproducibilidad y Gestión de Dependencias	44
9.3	Optimización del Pipeline de CI: Eficiencia y Velocidad	44
9.3.1	Estrategia de Caché Global con Fallback	44
9.3.2	Validación Basada en Revisión (PR-driven CI)	44
9.4	Integridad del Código y Gobernanza mediante Rulesets	44
9.5	Pruebas Automatizadas y Calidad de Código	45
9.6	Infraestructura como Código con Terraform	45
9.6.1	Pipeline de Despliegue	46
9.6.2	Módulos Terraform: Configuración Detallada	46
9.6.3	Especificaciones de la Instancia EC2	50
9.6.4	Visualización de Componentes AWS	51
9.6.5	Topología y Flujo de Datos	54
9.7	Despliegue Local: Kubernetes con ArgoCD y Kustomize	55
9.7.1	Arquitectura del Clúster Local	55
9.7.2	Flujo GitOps con ArgoCD	55
9.7.3	Ciclo de Despliegue Típico	56
9.7.4	Carga de Checkpoints y Uso Local	56
9.7.5	Ventajas del Despliegue Local	57
10	Conclusiones y Trabajo Futuro	58
10.1	Viabilidad de los SLM Híbridos	58
10.2	Lecciones Aprendidas	58
10.3	Trabajo Futuro	58
11	Anexo: Guía Rápida de Comandos	59
12	Anexo: Configuración del Backend	60

1 Introducción

1.1 Contexto y Motivación

Los Large Language Models (LLM) han transformado la inteligencia artificial moderna. Sin embargo, su desarrollo permanece concentrado en pocas organizaciones debido a costes prohibitivos: el entrenamiento de modelos de miles de millones de parámetros requiere infraestructura masiva y decenas de millones de dólares.

Los Small Language Models (SLM) emergen como alternativa viable para organizaciones que requieren modelos controlables, eficientes y especializados. A diferencia de depender de APIs de terceros o servicios en la nube, desarrollar un SLM internamente permite:

- Control total sobre datos y privacidad
- Optimización para hardware disponible (GPUs consumer-grade)
- Especialización en dominios específicos
- Reducción de latencia y costes de inferencia

El desafío reside en diseñar arquitecturas que capturen las capacidades de modelos grandes manteniendo eficiencia computacional.

1.2 Problema y Propuesta

Los Transformers puros, aunque efectivos, requieren múltiples capas de atención para capturar patrones secuenciales, incrementando VRAM y latencia. Las arquitecturas recurrentes (RNN, GRU) son eficientes pero pierden contexto a largo plazo.

Este trabajo propone una **arquitectura híbrida que combina Transformer Multi-Head Attention y capas GRU dentro de cada bloque**, permitiendo:

- Captura de dependencias globales vía Multi-Head Attention
- Refinamiento secuencial local vía GRU
- Reducción de requisitos de VRAM respecto a Transformers puros equivalentes

1.3 Objetivos

El objetivo principal es diseñar, implementar y validar esta arquitectura híbrida SLM, evaluando su rendimiento en términos absolutos y contextualizando los resultados respecto a la literatura publicada sobre modelos de escala comparable.

Objetivo 1 — Pre-entrenamiento y Validación Arquitectónica:

- Implementación de la arquitectura híbrida con optimizaciones CUDA
- Pre-entrenamiento desde cero sobre datasets conversacionales públicos (OpenAssistant, ShareGPT, Alpaca)

- Evaluación cuantitativa mediante métricas estándar de modelado del lenguaje:
 - Perplejidad (capacidad predictiva)
 - Peak VRAM (eficiencia de memoria)
 - Tokens/segundo (latencia de inferencia)
 - Precisión Top-1, Top-5 y Top-10

Objetivo 2 — Validación de Producción:

- Despliegue en Kubernetes con gestión automatizada mediante ArgoCD
- Análisis de cost-benefit: AWS Spot Instances vs desarrollo local
- Pipelines de testing automatizados para validación continua

1.4 Contribuciones

1. **Arquitectura híbrida Transformer-GRU:** demostración de que la combinación de atención global y recurrencia local permite reducir la huella de memoria sin sacrificar capacidad predictiva
2. **Benchmark exhaustivo:** evaluación cuantitativa de 10 métricas sobre 300.000 muestras de validación, con análisis de estabilidad y contextualización respecto a la literatura
3. **Pipeline de ingeniería end-to-end:** implementación completa desde pre-entrenamiento hasta despliegue en producción, incluyendo infraestructura cloud y automatización de testing

1.5 Estructura del Documento

- **Capítulo 2:** Estado del arte — evolución de LLM a SLM, RNN, LSTM, GRU, Transformers y arquitecturas híbridas modernas
- **Capítulo 3:** Fundamentos teóricos — formalización matemática del mecanismo de atención, MHA, GRU y análisis de complejidad híbrida
- **Capítulo 4:** Datos y datasets — descripción de los conjuntos de datos utilizados para el pre-entrenamiento
- **Capítulo 5:** Mejoras arquitectónicas e implementación — proceso iterativo de desarrollo y decisiones de ingeniería
- **Capítulo 6:** Diseño e implementación de la arquitectura híbrida — especificación técnica del modelo final
- **Capítulo 7:** Evaluación y benchmarking — 10 métricas de calidad, eficiencia y recursos sobre 300.000 muestras

- **Capítulo 8:** Infraestructura cloud y análisis de costes — provisión con Terraform, EC2, S3 y ECR
- **Capítulo 9:** MLOps: despliegue y puesta en producción — CI/CD, testing automatizado, Terraform y diseño GitOps con ArgoCD
- **Capítulo 10:** Conclusiones y trabajo futuro
- **Anexo A:** Guía rápida de comandos
- **Anexo B:** Configuración del backend

2 Estado del Arte

Este capítulo analiza el panorama actual de los modelos de lenguaje, la evolución hacia modelos más eficientes y las investigaciones previas en arquitecturas híbridas.

2.1 Evolución de los Modelos de Lenguaje: De LLM a SLM

En los últimos años, el campo del Procesamiento del Lenguaje Natural (NLP) ha estado dominado por los *Large Language Models* (LLM), cuya arquitectura base sigue siendo el Transformer propuesto por Vaswani et al. [1]. Modelos como Llama 2 [2] han demostrado capacidades emergentes sorprendentes al escalar el número de parámetros a miles de millones.

Sin embargo, recientemente ha surgido una tendencia hacia los *Small Language Models* (SLM), impulsada por la necesidad de eficiencia y despliegue en entornos con recursos limitados. Investigaciones como “*Textbooks Are All You Need*” [3] demuestran que la calidad de los datos puede compensar el tamaño del modelo, permitiendo que arquitecturas de menos de 3 mil millones de parámetros (como Phi-3 o Mistral) compitan en tareas específicas con modelos mucho más grandes. En este contexto, la familia de modelos SmolLM [4] representa el estado actual de la técnica en SLMs ultra-eficientes, optimizando tanto el proceso de entrenamiento como la curación de datos para maximizar el rendimiento en dispositivos de consumo.

2.2 Limitaciones de los Transformers Puros

A pesar de su éxito, los Transformers convencionales presentan un cuello de botella fundamental: la complejidad cuadrática del mecanismo de auto-atención respecto a la longitud de la secuencia [1]. Esto implica un consumo de memoria de vídeo (VRAM) y un tiempo de cómputo que crece de forma prohibitiva para contextos largos. En un entorno de producción, esto se traduce en altos costes operativos y latencias elevadas, lo que motiva la búsqueda de arquitecturas con escalado lineal.

2.3 Fundamentos: Redes Neuronales Recurrentes (RNN)

Las Redes Neuronales Recurrentes introducidas por Rumelhart et al. [?] representan la primera arquitectura capaz de procesar secuencias manteniendo un estado oculto a lo largo del tiempo. En su forma más simple, una RNN vanilla aplica la misma función de transformación recurrentemente:

$$h_t = \tanh(W_{ih}x_t + W_{hh}h_{t-1} + b) \quad (1)$$

donde h_t es el estado oculto en tiempo t , x_t es la entrada, y W_{ih} , W_{hh} son matrices de pesos. El estado oculto actúa como memoria, permitiendo que la red capture dependencias temporales.

Sin embargo, las RNNs vanilla sufren del problema de *vanishing gradients*: durante el backpropagation a través del tiempo (BPTT), los gradientes se atenúan exponencialmente al propagarse hacia atrás a través de muchos pasos temporales. Esto hace prácticamente imposible entrenar redes que capturen dependencias a largo plazo más allá de 10-20 pasos

de tiempo. El cálculo del gradiente requiere multiplicar matrices de Jacobiano repetidamente, y si sus valores propios son menores a 1, el gradiente tiende a cero exponencialmente.

2.4 LSTM y GRU: Soluciones a Dependencias Largo Plazo

Para resolver el problema de vanishing gradients, Hochreiter y Schmidhuber propusieron las *Long Short-Term Memory* (LSTM) [?] networks. Las LSTM introducen *compuertas* (gates) que controlan qué información fluye a través de la celda, permitiendo que gradientes fluyan sin atenuación:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (\text{forget gate}) \quad (2)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (\text{input gate}) \quad (3)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (\text{candidate memory}) \quad (4)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (\text{cell state}) \quad (5)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (\text{output gate}) \quad (6)$$

$$h_t = o_t \odot \tanh(C_t) \quad (7)$$

Las compuertas utilizan funciones sigmoideas que producen valores en $[0, 1]$, permitiendo multiplicación suave sin saturación. La célula de memoria C_t mantiene una conexión directa con gradientes a través de suma, evitando desvanecimiento.

Las *Gated Recurrent Units* (GRU) propuestas por Cho et al. [5] simplificaron este mecanismo combinando la compuerta de olvido e input en una sola compuerta de actualización. Las ecuaciones del GRU son:

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) \quad (\text{reset gate}) \quad (8)$$

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) \quad (\text{update gate}) \quad (9)$$

$$\tilde{h}_t = \tanh(W_h \cdot [r_t \odot h_{t-1}, x_t] + b_h) \quad (\text{candidate hidden}) \quad (10)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (11)$$

Las GRUs logran un rendimiento comparable a LSTMs con **menos parámetros** (33% menos) y son más fáciles de entrenar. Ambas arquitecturas eliminan el problema de vanishing gradients al mantener conexiones aditivas directas en la ruta del gradiente, pero siguen siendo secuenciales: el tiempo de inferencia crece linealmente con la longitud de la secuencia.

2.5 El Mecanismo de Atención y el Nacimiento de los Transformers

El cuello de botella de la secuencialidad se direccionó mediante el mecanismo de *atención*. Bahdanau et al. [?] propusieron que en lugar de comprimir toda la entrada en un único vector, el decodificador debería poder enfocarse selectivamente en diferentes partes de la entrada en cada paso de decodificación.

El mecanismo de atención calcula un score de relevancia entre una consulta (query) Q y un conjunto de claves (keys) K :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (12)$$

donde V son los valores. La matriz $\text{softmax}(QK^T/\sqrt{d_k})$ actúa como un mapa de pesos que distribuye atención sobre toda la secuencia. A diferencia de la recurrencia, el cálculo de atención es **paralelizable**: todos los pares de posiciones pueden computarse simultáneamente sin dependencias secuenciales.

Vaswani et al. [1] llevaron este concepto al extremo en su modelo *Transformer*, eliminando la recurrencia completamente y construyendo la arquitectura entera sobre mecanismos de auto-atención y redes feed-forward. El Transformer aplica múltiples cabezas de atención (Multi-Head Attention, MHA) en paralelo, permitiendo al modelo atender a diferentes subespacios de representación:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (13)$$

donde cada head_i es una atención independiente con sus propios pesos. La arquitectura resultante consta de:

1. **Capas de Atención:** Capturan dependencias globales a cualquier distancia en $O(n^2)$ complejidad de tiempo pero paralelizables completamente.
2. **Redes Feed-Forward:** Procesan cada posición independientemente con dos transformaciones lineales y una activación ReLU o GELU.
3. **Normalización de Capas:** Estabilizan el entrenamiento de redes profundas mediante normalización de activaciones.
4. **Conexiones Residuales:** Facilitan el flujo de gradientes a través de capas profundas.

La capacidad del Transformer de procesar en paralelo combinada con su capacidad de atender a cualquier token independientemente de su distancia revolucionó el NLP, permitiendo entrenar en datasets gigantescos y escalar a miles de millones de parámetros. Sin embargo, la complejidad cuadrática $O(n^2)$ del mecanismo de atención respecto a la longitud de secuencia n es prohibitiva para contextos largos.

2.6 Arquitecturas Híbridas y Recurrencia Moderna

Para mitigar las limitaciones del Transformer, han resurgido las arquitecturas basadas en recurrencia, pero con enfoques modernos. Las *Gated Recurrent Units* (GRU) [5] ofrecen un manejo eficiente de secuencias con un estado oculto de tamaño fijo, eliminando el coste cuadrático.

Investigaciones recientes como Mamba [6] han propuesto modelos de espacio de estados (*State Space Models*) que logran un rendimiento comparable a los Transformers pero con inferencia de tiempo lineal. La tendencia actual, y el núcleo de este TFM, es la hibridación:

combinar capas de atención para capturar relaciones globales con capas recurrentes o lineales para mantener la eficiencia secuencial.

Esta hibridación se ha visto facilitada por la evolución de los ecosistemas de software, específicamente con la transición de la librería *Transformers* de Hugging Face hacia la versión 5. Mientras que las versiones precedentes (v4.x) requerían implementaciones manuales complejas para integrar componentes recurrentes en el grafo de computación del Transformer, la versión 5 introduce abstracciones nativas para modelos de espacio de estados y arquitecturas secuenciales no atencionales. Estas mejoras permiten una gestión optimizada de la caché de inferencia (KV Cache) y una integración transparente de capas GRU dentro del flujo autorregresivo, garantizando que el modelo mantenga la compatibilidad con herramientas de optimización y despliegue estándar del sector.

3 Fundamentos Teóricos

En este capítulo se formalizan matemáticamente las tecnologías base que componen la propuesta híbrida. El objetivo es doble: por un lado, establecer el lenguaje técnico preciso que se utilizará en los capítulos de implementación y evaluación; por otro, justificar mediante análisis de complejidad la elección de una arquitectura mixta frente a los enfoques puros predominantes en la industria.

3.1 El Mecanismo de Atención

El mecanismo de *self-attention*, introducido por Vaswani et al. [1], opera sobre una secuencia de L vectores de entrada $X \in \mathbb{R}^{L \times d_{model}}$, proyectándolos en tres espacios distintos mediante matrices aprendibles:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \quad (14)$$

donde $W_Q, W_K, W_V \in \mathbb{R}^{d_{model} \times d_k}$ son las matrices de proyección para *queries*, *keys* y *values* respectivamente. La salida de atención se obtiene como:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (15)$$

El factor de escala $\frac{1}{\sqrt{d_k}}$ previene que el producto interior $QK^\top$ crezca en magnitud con la dimensión, evitando regiones de gradiente muy pequeño en la función softmax. La matriz resultante $QK^\top \in \mathbb{R}^{L \times L}$ es el núcleo del problema de escalabilidad: su cómputo requiere $O(L^2)$ operaciones y su almacenamiento $O(L^2)$ memoria, lo que limita severamente el procesamiento de contextos largos en entornos con VRAM reducida.

3.2 Multi-Head Attention

El mecanismo de *Multi-Head Attention* (MHA) extiende la atención simple ejecutando h cabezas en paralelo, cada una con sus propias proyecciones de dimensión reducida $d_k = d_{model}/h$:

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O \quad (16)$$

$$\text{head}_i = \text{Attention}(QW_{Q_i}, KW_{K_i}, VW_{V_i}) \quad (17)$$

En el modelo propuesto se utilizan $h = 12$ cabezas sobre un espacio de $d_{model} = 768$ dimensiones, resultando en $d_k = 64$ por cabeza. Esta configuración captura relaciones semánticas a distintos niveles de abstracción simultáneamente, desde co-referencias pronominales hasta dependencias sintácticas de largo alcance.

3.3 Gated Recurrent Units (GRU)

Las *Gated Recurrent Units* [5] son una variante simplificada de las LSTM [7] diseñada para reducir el número de puertas sin sacrificar capacidad de memoria selectiva. Dado un token de entrada x_t y el estado oculto anterior h_{t-1} , el GRU computa:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad (\text{puerta de actualización}) \quad (18)$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad (\text{puerta de reinicio}) \quad (19)$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h) \quad (\text{candidato de estado}) \quad (20)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (\text{estado oculto}) \quad (21)$$

donde σ es la función sigmoide y $\odot$ denota el producto elemento a elemento. La **puerta de actualización** z_t decide qué fracción del estado anterior se conserva; la **puerta de reinicio** r_t controla cuánta información del pasado es relevante para computar el candidato $\tilde{h}_t$.

3.3.1 Justificación de GRU frente a LSTM

La elección de GRU sobre LSTM está motivada por criterios de eficiencia en parámetros: una celda GRU tiene dos puertas frente a las tres de una LSTM (más la celda de memoria c_t), lo que reduce el número de parámetros por unidad en aproximadamente un 25%. Estudios empíricos como los de Chung et al. [8] demuestran que esta reducción no implica pérdida significativa de rendimiento en tareas de modelado del lenguaje cuando el estado oculto tiene dimensión suficiente. En el contexto de un SLM con recursos limitados, esta eficiencia es determinante.

3.4 La Hipótesis Híbrida: Análisis de Complejidad

La motivación central de este trabajo es la superación del paradigma de “*atención pura*” para modelos de escala reducida. La arquitectura híbrida propuesta se fundamenta en un análisis de complejidad comparativo entre los tres enfoques posibles:

Arquitectura	Memoria	Cómputo	Estado secuencial
Transformer puro	$O(L^2)$	$O(L^2 d)$	KV Cache (crece con L)
RNN / GRU puro	$O(d)$	$O(L d^2)$	Estado fijo $h \in \mathbb{R}^d$
Híbrido (propuesto)	$O(L^2 + d)$	$O(L^2 d + L d^2)$	KV Cache + estado GRU

Table 1: Comparativa de complejidad computacional y de memoria.

El diseño híbrido no mejora la complejidad asintótica en el peor caso, pero ofrece ventajas prácticas relevantes para el rango de contextos cortos-medios ($L \leq 1024$) en los que opera este modelo:

1. **Eficiencia de memoria por bloque:** el GRU actúa como un *compresor de contexto*, manteniendo un estado interno fijo $h \in \mathbb{R}^d$ que resume el historial de la secuencia sin necesidad de almacenar las activaciones intermedias de todos los tokens anteriores.
2. **Separación de responsabilidades:** la atención captura dependencias globales de largo alcance, mientras el GRU modela la estructura secuencial local (orden de palabras, concordancia morfológica). Esta división evita que un único mecanismo deba resolver ambas escalas, reduciendo la carga efectiva de cada componente.

3. **Inferencia estable en contextos largos:** el estado oculto del GRU permanece acotado independientemente de L , proporcionando una representación del historial con latencia constante que complementa el KV Cache de atención.

4 Datos y Datasets para Pre-entrenamiento

El rendimiento de un Small Language Model depende críticamente de la calidad, diversidad y representatividad de los datos de entrenamiento. Este capítulo documenta exhaustivamente la estrategia de curación, selección y procesamiento de datasets que alimentan el modelo híbrido propuesto, justificando cada decisión arquitectónica sobre la base de las características semánticas y lingüísticas de cada fuente.

4.1 Principios de Diseño para la Mezcla de Datos

La selección de datasets para pre-entrenamiento de un SLM requiere equilibrar múltiples objetivos en tensión:

1. **Cobertura Lingüística:** El modelo debe capturar patrones sintácticos y semánticos amplios, desde gramática básica hasta construcciones complejas.
2. **Diversidad de Dominios:** Exposición a vocabulario técnico, conversacional, literario e instructivo evita sobreespecialización.
3. **Naturalidad del Lenguaje:** Preferencia por diálogos y textos producidos por humanos sobre datos sintéticos, aunque estos últimos puedan complementar.
4. **Eficiencia Computacional:** Bajo costo de descargar, procesar y almacenar en entornos con recursos limitados.

La estrategia adoptada en este TFM selecciona cuatro datasets de acceso abierto de Hugging Face, cada uno contribuyendo capacidades complementarias. Un quinto dataset (The Stack) se mantiene comentado debido a restricciones de acceso.

4.2 Descripción Detallada de Datasets

4.2.1 OpenAssistant (OASST1)

Identificador: `OpenAssistant/oasst1` [?]

Provenance: Comunidad abierta de voluntarios, financiado por Open Source Initiative

Tipo de Datos: Diálogos multi-turno conversacionales

OpenAssistant (OASST1) es una iniciativa comunitaria que recopila diálogos naturales entre usuarios y asistentes de IA. Cada conversación contiene múltiples turnos con preguntas, respuestas y retroalimentación de calidad (ratings de utilidad, veracidad y calidad).

Características Principales:

- **Número de ejemplos:** 161k conversaciones (664k turnos totales)
- **Tono:** Informal, conversacional; refleja interacciones reales de humanos con sistemas IA
- **Cobertura:** Preguntas de factibilidad, razonamiento, matemáticas elementales, creatividad

- **Longitud promedio:** 200-500 tokens por turno
- **Idiomas:** Principalmente inglés, con versiones en alemán, francés, español, japonés

Rol en el Modelo: OASST1 proporciona la base de naturalidad conversacional. El modelo aprende a reconocer estructura de turnos (turno de usuario, turno de asistente), mantener coherencia entre turnos sucesivos, y adaptarse a estilos de preguntas variados. Esta fuente es crítica para evitar lenguaje robótico o excesivamente formal que limitaría la aplicabilidad a casos de uso reales.

4.2.2 ShareGPT

Identificador: anon8231489123/ShareGPT_Vicuna_unfiltered [?]

Provenance: Transcripciones de interacciones con GPT-3.5-Turbo publicadas por usuarios

Tipo de Datos: Diálogos multi-turno con estilos de respuesta avanzados

ShareGPT es un dataset de interacciones reales con modelos de lenguaje grandes (principalmente GPT-3.5-Turbo), recopiladas por usuarios y compartidas públicamente. A diferencia de OASST1, ShareGPT contiene respuestas de modelos más sofisticados, permitiendo que el modelo entrenado transfiera patrones resolutivos de sistemas maduros.

Características Principales:

- **Número de ejemplos:** 90k conversaciones
- **Tono:** Más formal y estructurado que OASST1; refleja estilos de respuesta de GPT-3.5
- **Cobertura:** Escritura técnica, análisis, generación creativa, resolución de problemas
- **Longitud promedio:** 300-1000 tokens por turno (respuestas más largas que OASST1)
- **Complejidad:** Mayor nivel de razonamiento y elaboración en las respuestas

Rol en el Modelo: ShareGPT actúa como canal de transferencia de estilos resolutivos. Aunque el modelo no es explícitamente destilado de GPT-3.5, la exposición a respuestas bien estructuradas y razonadas permite que desarrolle patrones similares de profundidad y cobertura de problemas. Esto es especialmente importante para tareas donde es necesario demostrar razonamiento paso a paso.

4.2.3 Alpaca

Identificador: tatsu-lab/alpaca [?]

Provenance: Conjunto de 52k ejemplos generados mediante instruction-tuning con GPT-3.5-Turbo

Tipo de Datos: Pares instruction-output estructurados

Alpaca es un dataset de 52k ejemplos creado mediante destilación de instrucciones. Cada ejemplo consta de una instrucción (ej: “Clasifica el siguiente párrafo según su sentimiento”) y su output correspondiente. A diferencia de los datasets conversacionales

anteriores, Alpaca proporciona una base clara de instrucciones y ejecuciones, facilitando el aprendizaje de tareas específicas.

Características Principales:

- **Número de ejemplos:** 52k pares instruction-output
- **Estructura:** Campo `instruction` (tarea a realizar), `input` (contexto opcional), `output` (respuesta esperada)
- **Cobertura:** Clasificación, extracción, generación, análisis, matemáticas
- **Longitud promedio:** 100-300 tokens por instrucción, 200-500 tokens por output
- **Característica única:** Incluye un campo `input` que permite contextualizaciones; muchos ejemplos tienen input vacío

Rol en el Modelo: Alpaca proporciona señal de entrenamiento estructurada para instruction-following. A través de este dataset, el modelo aprende a reconocer instrucciones explícitas y a responder de forma directa y organizada. Esto es crítico para tareas donde la claridad y la estructura de la respuesta son más importantes que la naturalidad conversacional.

4.2.4 UltraChat

Identificador: `stingning/ultrachat` [?]

Provenance: 1.4M ejemplos generados mediante destilación de Turbo con contextos largos

Tipo de Datos: Diálogos multi-turno con énfasis en consistencia a largo plazo

UltraChat es el dataset más grande de la mezcla, enfatizando conversaciones largas y consistentes. Mientras que OASST1 y ShareGPT contienen diálogos con número de turnos variable, UltraChat fue diseñado específicamente para entrenar modelos capaces de mantener coherencia en contextos extendidos (hasta 1000+ tokens).

Características Principales:

- **Número de ejemplos:** 1.4M ejemplos (dataset más grande de la mezcla)
- **Longitud de conversación:** 8-20 turnos por diálogo típicamente
- **Longitud de contexto:** Contextos acumulativos de hasta 1000-2000 tokens
- **Tono:** Similar a ShareGPT; respuestas bien estructuradas y razonadas
- **Propósito original:** Entrenar modelos para tareas que requieren mantener estado conversacional complejo

Rol en el Modelo: UltraChat es fundamental para validar la arquitectura híbrida propuesta. Mientras que los Transformers puros pueden procesar contextos largos mediante su mecanismo de atención global, la arquitectura híbrida combina atención con GRU para mantener memoria secuencial. El volumen de datos de contexto largo de UltraChat permite que el modelo practique tanto el mecanismo de atención (capturando relaciones globales) como el GRU (refinando patrones secuenciales), entrenando efectivamente la verdadera hibridación arquitectónica.

4.2.5 The Stack (YAML) - Gated, No Incluido

Identificador: bigcode/the-stack [?] (subconjunto: data/yaml)

Estado: Comentado en configuración; requiere aprobación manual en Hugging Face

Tipo de Datos: Archivos de configuración YAML, Dockerfiles, scripts de infraestructura

The Stack es un corpus gigantesco (11 terabytes de código fuente) compilado a partir de repositorios públicos. El subconjunto YAML contiene archivos de configuración y manifests de infraestructura, útiles para dotar al modelo de comprensión de sintaxis DevOps y patrones de infraestructura-como-código.

Características Principales:

- **Acceso:** Dataset gated; requiere aceptación de términos y aprobación en Hugging Face
- **Tamaño total:** 11 TB; subconjunto YAML es 50 GB
- **Contenido:** YAML, JSON, Docker, Kubernetes, Terraform, configuraciones varias
- **Propósito:** Conocimiento especializado en infraestructura y configuración

Razón de Exclusión: Aunque técnicamente valuable, la naturaleza gated del dataset (requiere aprobación explícita en Hugging Face) añade fricción operacional. Para este TFM, se priorizó la reproducibilidad inmediata con datasets de acceso abierto. Sin embargo, The Stack puede integrarse fácilmente una vez aprobado.

4.3 Estrategia de Mezcla: Ponderación y Proporción

Cada dataset contribuye una proporción diferente al entrenamiento, diseñada para equilibrar cobertura y eficiencia:

Dataset	Peso	Ejemplos	Justificación
OpenAssistant	30%	30k ejemplos	Base de naturalidad conversacional
UltraChat	30%	30k ejemplos	Validación de consistencia largo plazo y arquitectura híbrida
Alpaca	20%	10.4k ejemplos	Señal de instruction-following estructurado
ShareGPT	20%	18k ejemplos	Transferencia de estilos resolutivos avanzados
Total	100%	88.4k	Equilibrio entre naturalidad, estructura y complejidad

Table 2: Distribución de pesos para mezcla de datasets de pre-entrenamiento. Con 100k ejemplos totales, cada época consume 88.4k muestras de los datasets disponibles.

4.3.1 Justificación de Pesos

Los pesos se eligieron mediante análisis de sesgo-varianza:

1. **OpenAssistant + UltraChat (60% combinado):** Dos tercios de los datos provienen de conversaciones naturales. Esto garantiza que el modelo desarrolle

un núcleo sólido de comprensión conversacional sin ser arrastrado hacia respuestas excesivamente formales o sintéticas.

2. **Alpaca (20%)**: Una quinta parte dedicada a instruction-following proporciona señal clara de tareas sin abrumar el modelo. Demasiado Alpaca (¡33%) haría el modelo rígido; demasiado poco (¡10%) limitaría su capacidad de seguimiento de instrucciones.
3. **ShareGPT (20%)**: Una quinta parte dedicada a respuestas complejas y bien razonadas facilita transferencia de estilos avanzados sin sobredominancia. Este equilibrio permite que el modelo desarrolle profundidad de razonamiento mientras mantiene la naturalidad de OpenAssistant.

El peso de UltraChat (30%) es especialmente importante para esta arquitectura: mientras que un Transformer puro podría beneficiarse menos de contextos largos (por el costo cuadrático), la arquitectura híbrida con GRU está optimizada para explotar memoria secuencial eficiente. UltraChat permite que el modelo entrene esta capacidad diferencialmente.

4.4 Procesamiento de Datos y Formato Apache Arrow

4.4.1 Pipeline de Descargar y Procesamiento

La infraestructura de datos implementa un pipeline automatizado en tres fases:

Fase 1: Descarga (DatasetDownloader)

El módulo `app/dataset/downloader.py` descarga cada dataset desde Hugging Face mediante la librería `datasets`. El descargador es **idempotente**: antes de descargar, verifica si el dataset ya existe localmente en la caché. Esto evita downloads redundantes durante ciclos de desarrollo iterativos.

```
1 downloader = DatasetDownloader(output_dir=".datasets")
2 downloader.download_single("open_assistant", config)
3 # Checkea existencia local; solo descarga si falta
```

Listing 1: Descarga idempotente de datasets

Fase 2: Armonización (DatasetProcessor)

Los datasets descargados tienen esquemas heterogéneos. OpenAssistant tiene estructura JSON anidada, Alpaca usa campos `instruction/input/output`, UltraChat usa `data` como lista de strings. El módulo `app/dataset/processor.py` normaliza todos los datasets a un esquema común con una columna `text`:

```
1 def harmonize_dataset(ds: Dataset, key: str) -> Dataset:
2     if key == "ultrachat":
3         # UltraChat: concatenar lista de strings
4         return ds.map(lambda x: {"text": "\n".join(x["data"])})
5     elif key == "alpaca":
6         # Alpaca: concatenar instruction + input + output
7         return ds.map(lambda x: {"text": format_alpaca(x)})
8     # ... (similar para otros)
```

Listing 2: Armonización de esquemas heterogéneos

Fase 3: Serialización a Apache Arrow

Una vez armonizado, todo se tokeniza y serializa a formato Apache Arrow para acceso eficiente:

```

1 tokenized_ds.save_to_disk(".datasets/mixed_dataset")
2 # Resultado: archivos .arrow con metadata JSON

```

Listing 3: Serialización a Arrow con tokenización

4.4.2 Ventajas del Formato Apache Arrow

1. **Memory Mapping (Zero-Copy Reading):** Los archivos `.arrow` se mapean directamente a la memoria RAM sin copiar datos. Esto permite entrenar con datasets de decenas de gigabytes usando máquinas con VRAM limitada (16 GB). El kernel del SO maneja el paging automático, trayendo datos del disco bajo demanda.
2. **Almacenamiento Columnar:** A diferencia de JSON (por fila) o Parquet (variabilidad), Arrow almacena por columnas. Para datasets de NLP donde cada ejemplo tiene un único campo `text`, esto optimiza compresión y velocidad de lectura secuencial.
3. **Integración Nativa con Transformers:** La librería `transformers` de Hugging Face y `datasets` están construidas sobre Arrow internamente. Usar Arrow evita conversiones y permite que el framework optimice automáticamente la lectura en batches.
4. **Tipificación Estricta:** Arrow impone esquemas tipificados (integer, string, float, etc.). Esto previene errores silenciosos donde un valor inesperado rompe downstream sin advertencia clara.

4.5 Estadísticas Finales de Entrenamiento

Con la mezcla y ponderación descritas, el dataset consolidado tiene las siguientes características:

Métrica	Descripción	Valor
Ejemplos Totales	Suma ponderada de ejemplos en mezcla	88,400
Tokens por Ejemplo	Promedio; rango 100-2000	850 tokens
Contexto Máximo	Longitud máxima procesada por modelo	1024 tokens
Vocabulario	Tokens únicos (GPT-2 tokenizer)	50,257
Épocas de Entrenamiento	Ciclos completos sobre el dataset	5 épocas
Total Tokens Procesados	$88.4k \times 850 \times 5$	375M tokens

Table 3: Estadísticas resumidas del dataset consolidado de entrenamiento. Con 375M tokens procesados en 5 épocas y modelo de 124M parámetros, la relación tokens/parámetros es 3:1, dentro del rango óptimo reportado en literatura (2-10).

Esta relación de 3:1 es significativa: investigaciones recientes (“Chinchilla Scaling Laws”, “Textbooks Are All You Need”) sugieren que la mejor eficiencia se logra con datos cuantitativamente moderados pero cualitativamente superiores. El ratio de nuestro SLM refleja una apuesta consciente por calidad (mezcla cuidadosa de 4 datasets diversos) sobre cantidad bruta.

4.6 Validación de Cobertura y Diversidad

Para validar que la mezcla logra el objetivo de diversidad, el pipeline incluye análisis post-procesamiento:

1. **Análisis de Vocabulario:** Histograma de tokens únicos por dataset para garantizar que ningún dataset domina en cobertura léxica.
2. **Análisis de Longitud:** Distribución de longitudes de ejemplo; si están todas agrupadas (ej: todas cortas), indica falta de diversidad.
3. **Análisis de Dominios:** Manual inspection de samples aleatorios para verificar que cubren conversación, instrucciones, razonamiento, técnica.

Estos análisis se documentan en la suite de tests automatizados (Capítulo ??).

5 Mejoras Arquitectónicas e Implementación del Modelo Híbrido

El diseño de un Small Language Model basado en una arquitectura híbrida requiere consideraciones cuidadosas sobre cómo integrar efectivamente dos paradigmas computacionales distintos: la atención global de los Transformers y la memoria secuencial eficiente de las unidades recurrentes. Este capítulo documenta el proceso iterativo de desarrollo: qué limitaciones presentaba la propuesta inicial y qué decisiones de ingeniería se tomaron para superarlas. La arquitectura resultante de este proceso se describe formalmente en el Capítulo 6.

5.1 Arquitectura Original y sus Limitaciones

La propuesta inicial de una arquitectura híbrida combinaba Transformers y GRU en cada bloque, pero presentaba una falla fundamental: los estados ocultos del GRU se descartaban al final de cada capa, eliminando la capacidad de memoria recurrente real que justificaba su inclusión. En el flujo original:

1. El GRU procesaba la entrada y generaba un estado oculto
2. Este estado oculto se descartaba inmediatamente
3. En el siguiente bloque, el GRU volvía a inicializar desde cero
4. Como resultado, no había información compartida verticalmente (entre bloques) a través del componente recurrente

Además, el GRU procesaba *después* de la atención, causando redundancia: la atención ya había capturado dependencias globales, y el GRU intentaba refinar patrones secuenciales sobre una representación ya modificada. Finalmente, sin control explícito de gradientes, el modelo sufría de explosiones típicas de arquitecturas recurrentes durante el entrenamiento temprano.

5.2 Mejora 1: Persistent GRU Hidden States (CRÍTICA)

La primera y más importante mejora es hacer que los estados ocultos del GRU sean **persistentes** a través de todos los 12 bloques del modelo. En lugar de descartar el hidden state al final de cada bloque, ahora se mantiene un tensor de hidden states (uno por bloque) que fluye a través de la red durante todo el forward pass.

5.2.1 Implementación Técnica

En la clase `HybridBlock`, el método `forward` fue reescrito para retornar tanto la salida como el nuevo hidden state. Cada bloque mantiene conexiones residuales que aseguran que tanto la información de atención como la del GRU se preservan y combinan:

- **Pre-LayerNorm**: Normalización aplicada antes de cada sub-componente para estabilidad numérica

- **GRU Processing:** Captura patrones secuenciales locales, mantiene estructura sintáctica
- **Attention Refining:** Refina con contexto global, integrando información de todo el contexto
- **MLP Refinement:** Procesamiento de características no-lineal final con factor de expansión 4

En la clase `HybridModel`, el forward pass ahora mantiene un registro de estos estados a través de todos los bloques. La inicialización de estos estados es crítica: cuando se comienza el entrenamiento, todos los hidden states se inicializan como `None`, permitiendo que el GRU comience sin sesgo previo. Sin embargo, después del primer forward pass, estos estados contienen información aprendida que se propaga al siguiente paso.

5.2.2 Impacto en la Memoria Recurrente

Con esta mejora, cada bloque GRU ahora **recuerda** información del bloque anterior. Si el Bloque 1 identifica un patrón secuencial importante (ej: estructura sintáctica de una frase), ese patrón se representa en su hidden state y se propaga al Bloque 2. El Bloque 2 puede entonces usar esa información para refinar su comprensión local, creando una verdadera arquitectura recurrente vertical.

Esto es conceptualmente diferente de una RNN pura, donde la información fluye solo horizontalmente (a través de la secuencia). Aquí, la información fluye tanto horizontalmente (dentro de cada bloque) como verticalmente (entre bloques a través de GRU), proporcionando una forma de “corto circuito recurrente” que complementa la atención. Este flujo de información vertical es lo que permite que una arquitectura con solo 12 capas (versus 30 en SmolLM2) pueda mantener poder expresivo comparable.

5.3 Mejora 2: Reorden de Componentes (GRU Pre-Attention)

La segunda mejora arquitectónica cambia el orden de procesamiento dentro de cada bloque. Originalmente, el GRU procesaba *después* de la atención. Esto es subóptimo porque:

- La atención ya ha transformado la representación radicalmente
- El GRU intenta capturar patrones secuenciales sobre una entrada altamente modificada
- Hay redundancia: ambos componentes están intentando capturar la misma información
- La atención puede perder información local importante que el GRU necesita

La nueva arquitectura posiciona el GRU **antes** de la atención, creando un flujo procesamiento que va de lo local a lo global:

1. **GRU (primero):** Refina patrones secuenciales locales, mantiene estructura sintáctica, procesa basándose en el hidden state del bloque anterior

2. **Attention** (segundo): Refina con contexto global, integrando información de todo el contexto, ahora trabajando sobre una representación ya parcialmente refinada
3. **MLP** (tercero): Procesamiento de características no-lineal final con expansión a 3072 dimensiones

Este orden permite que la atención trabaje sobre una representación que ya ha sido refinada localmente, en lugar de sobre la entrada bruta o una salida altamente procesada. La información fluye desde lo local a lo global, reflejando mejor cómo pensaríamos que debe procesarse el lenguaje: primero capturando relaciones locales (orden de palabras, concordancia), luego refinando con contexto global (referencia anafórica, semántica de largo alcance).

5.4 Mejora 3: Inicialización Ortogonal para Componentes RNN

Las unidades recurrentes son notoriamente sensibles a la inicialización de pesos. Una inicialización inadecuada puede llevar a vanishing gradients o exploding gradients durante el backpropagation a través del tiempo (BPTT).

La mejora implementa inicialización ortogonal para los pesos del GRU, mientras mantiene inicialización normal estándar para componentes no-recurrentes. Específicamente:

- Pesos `weight_ih` (input-to-hidden): inicializados como matriz ortogonal
- Pesos `weight_hh` (hidden-to-hidden): inicializados como matriz ortogonal
- Bias: inicializados a cero
- Todos los demás módulos (Linear, Embedding, LayerNorm): inicialización estándar

La inicialización ortogonal preserva la magnitud de los vectores a través de las transformaciones lineales del GRU, lo que reduce el riesgo de vanishing/exploding gradients. Las matrices ortogonales tienen la propiedad de que $Q^T Q = I$, lo que significa que la transformación preserva normas y ángulos. Para redes recurrentes, esto es crítico porque permite que gradientes fluyan sin atenuación o amplificación artificial a través de múltiples pasos de tiempo.

5.5 Mejora 4: Gradient Clipping (CRÍTICA)

Aunque la inicialización ortogonal ayuda, las redes recurrentes siguen siendo propensas a explosiones de gradiente. Por ello, se implementó un mecanismo de clipping de gradientes que limita la norma de los gradientes durante el entrenamiento.

El clipping se aplica después de cada backward pass y antes del optimizer step, asegurando que ningún gradiente explosión pueda causar actualizaciones de parámetros descontroladas. El parámetro `max_norm` (típicamente 1.0) define el umbral máximo. Si la norma de gradientes excede este valor, todos los gradientes se escalan proporcionalmente hacia abajo.

Matemáticamente, si $\|\nabla L\| > \text{max_norm}$, entonces $\nabla L \leftarrow \nabla L \cdot \frac{\text{max_norm}}{\|\nabla L\|}$.

Este método es especialmente importante en arquitecturas híbridas donde el GRU puede amplificar gradientes a través de sus múltiples iteraciones internas. Sin clipping,

un paso de tiempo con gradientes grandes puede causar que todos los 12 bloques GRU amplifiquen ese gradiente, llevando a explosión. Con clipping, esta amplificación es limitada.

5.6 Mejora 5: Logging de Contribuciones de Componentes

Para entender cómo cada componente (GRU, Attention, MLP) contribuye al entrenamiento, se implementó un sistema de logging que rastrea la norma de gradientes para cada uno en tiempo real.

Durante cada paso de entrenamiento, después del backward pass y clipping, se calcula:

- **GRU norm:** $\sqrt{\sum_{\text{params in GRU}} (\nabla param)^2}$
- **Attention norm:** $\sqrt{\sum_{\text{params in Attention}} (\nabla param)^2}$
- **MLP norm:** $\sqrt{\sum_{\text{params in MLP}} (\nabla param)^2}$
- **GRU/Attention ratio:** $\frac{\text{GRU norm}}{\text{Attention norm} + \epsilon}$

El ratio GRU/Attention es la métrica clave. Permite validar que la arquitectura es verdaderamente híbrida:

- **Ratio ¡ 0.3:** GRU demasiado débil, indica que la atención domina completamente, el GRU no está aprendiendo
- **Ratio 0.5-1.5: BALANCEADO**, ambos componentes contribuyen significativamente al aprendizaje
- **Ratio ¿ 1.5:** GRU dominando, indica desbalance donde GRU absorbe la mayoría del aprendizaje

El logging se produce cada N pasos configurables (típicamente 100), permitiendo monitoreo en tiempo real. Un log típico se ve como:

```
[Step 100] Loss: 4.2341 | GRU: 0.4521 | Attn: 0.3841 | MLP: 0.2154 | Ratio: 1.176 | GradNorm: 2.341->1.000
```

5.7 Mejora 6: Validation Loop

Para detectar overfitting temprano y validar que el modelo está aprendiendo efectivamente, se implementó una evaluación periódica en un conjunto de validación durante el entrenamiento.

La validation loop se ejecuta cada N pasos (típicamente 500), evaluando el modelo en un mini-batch de validación. Esto permite:

- Detectar divergencia entre training loss y validation loss (indicador de overfitting)
- Validar que el entrenamiento es estable

- Tomar decisiones sobre cuándo detener el entrenamiento

Se espera que durante el entrenamiento correcto, tanto training loss como validation loss disminuyan. Si validation loss comienza a aumentar mientras training loss sigue bajando, indica overfitting en los datos de entrenamiento.

5.8 Mejora 7: Herramienta de Análisis Arquitectónico

Se creó la clase `HybridArchitectureAnalyzer` que proporciona capacidades de diagnóstico detalladas más allá del logging básico. Esta herramienta permite:

- Análisis completo de contribuciones de componentes
- Exportación de métricas a JSON para análisis posterior con herramientas externas
- Generación de reportes legibles por humanos
- Análisis del flujo de hidden states a través de capas
- Comparación de arquitecturas alternativas

El analyzer puede ser invocado durante o después del entrenamiento, permitiendo tanto monitoreo en vivo como análisis post-hoc de los datos de entrenamiento.

5.9 Mejora 8: Suite de Tests Automatizados

Para garantizar la corrección de la implementación, se creó una suite de 7 tests automatizados que validan cada aspecto de la arquitectura:

1. **Forward Pass:** Verifica que el output tiene las dimensiones correctas
2. **Loss Computation:** Valida que la pérdida se calcula correctamente
3. **Gradient Flow:** Asegura que gradientes fluyen correctamente a través de todos los componentes
4. **Hidden State Persistence:** Verifica que los hidden states se mantienen a través de bloques
5. **Weight Initialization:** Confirma que los pesos se inicializan correctamente
6. **Component Analysis:** Valida que el analyzer funciona correctamente
7. **Model Size:** Confirma que el conteo de parámetros es el esperado

Ejecutar los tests:

```
python test_improvements.py
```

5.10 Impacto Cuantificable en la Convergencia

Las mejoras arquitectónicas tienen impacto medible y significativo en la convergencia del modelo durante el entrenamiento desde cero.

5.10.1 Velocidad de Convergencia

- **Arquitectura Original:** Loss 5.4 $\rightarrow$ 5.1 $\rightarrow$ 4.8 (primeros 300 steps)
- **Arquitectura Mejorada:** Loss 5.4 $\rightarrow$ 4.9 $\rightarrow$ 4.3 (primeros 300 steps)
- **Mejora:** 15% más rápido en convergencia inicial

Esta mejora en velocidad es significativa porque acelera todo el ciclo de entrenamiento. Con 5 épocas de entrenamiento, una mejora del 15% en convergencia temprana se traduce en horas ahorradas.

5.10.2 Estabilidad de Gradientes

- **Arquitectura Original:** Gradient spikes erráticos, máximo 45+, completamente sin control
- **Arquitectura Mejorada:** Controlados mediante clipping en rango [0.5, 1.0]
- **Mejora:** 45 $\times$ más estable, entrenamiento predecible y reproducible

La estabilidad es crítica para evitar colapso del entrenamiento. Sin ella, el modelo puede alcanzar estados no-recuperables donde los parámetros divergen.

5.10.3 Visibilidad y Debuggabilidad

- **Arquitectura Original:** Solo visible loss total, imposible saber si componentes funcionan correctamente
- **Arquitectura Mejorada:** GRU norm, Attention norm, MLP norm, ratio cada 100 steps, completamente transparente
- **Mejora:** Arquitectura totalmente transparente y debuggeable

5.11 Configuración Recomendada para Entrenamiento

Basado en las mejoras implementadas, la configuración recomendada para entrenar el modelo es:

```
from app.training.trainer import TrainingService

service = TrainingService(dataset_path=".datasets/mixed_dataset")

service.train(
    epochs=5,
```

```
batch_size=64,  
learning_rate=1e-4,  
grad_clip_norm=1.0,          # Control de gradientes  
log_metrics_every=100,       # Log de métricas  
validate_every=500,          # Validación periódica  
)
```

Con esta configuración:

- Los gradientes se mantienen controlados mediante clipping
- Las métricas de cada componente se registran regularmente para monitoreo
- La validación detecta overfitting temprano
- Los hidden states del GRU persisten y contribuyen significativamente al aprendizaje

Las mejoras descritas en este capítulo consolidan el proceso de desarrollo de la arquitectura. El diseño final resultante, con sus especificaciones técnicas completas y la estrategia de entrenamiento, se detalla en el capítulo siguiente.

6 Diseño e Implementación de la Arquitectura Híbrida

Este capítulo detalla la aportación técnica central de este TFM: la integración de Transformers y GRU en un único modelo funcional, así como la estrategia de datos y fine-tuning que permite su entrenamiento eficiente y la comparativa con el estado del arte.

6.1 Estrategia de Curación y Mezcla de Datos (Data Mixing)

La calidad del entrenamiento de un Small Language Model (SLM) depende críticamente de la diversidad y representatividad de los datos. Se ha diseñado una estrategia de *Data Mixing* para la fase de pre-entrenamiento y alineación general, reservando el conocimiento específico para una fase final.

6.1.1 Selección de Datasets de Propósito General

Se han seleccionado cinco pilares de datos abiertos para dotar al modelo de capacidades lingüísticas, de razonamiento y técnicas:

- **OpenAssistant (OASST1):**¹ Fuente principal de naturalidad, proporcionando turnos de diálogo con matices humanos reales.
- **ShareGPT:**² Utilizado para transferir el estilo resolutorio y la estructura de respuesta de modelos avanzados.
- **Alpaca:**³ Aporta una base sólida de *instruction-tuning* para tareas directas.
- **UltraChat:**⁴ Fundamental para garantizar la consistencia en diálogos largos, poniendo a prueba la memoria de la arquitectura híbrida.
- **The Stack (YAML):**⁵ Subconjunto del dataset *The Stack* filtrado para archivos de configuración YAML, esencial para el conocimiento de infraestructura y sintaxis DevOps.

6.1.2 Infraestructura de Datos: Formato Apache Arrow

Para garantizar un acceso eficiente a los datasets durante el entrenamiento, se ha optado por el uso de archivos en formato **Apache Arrow** (`.arrow`). Este estándar, que subyace a la librería *datasets* de Hugging Face utilizada en el proyecto, ofrece ventajas tecnológicas clave para la viabilidad de los SLMs:

- **Carga mediante Memory Mapping:** Los archivos `.arrow` permiten el acceso de tipo “zero-copy”, mapeando los datos directamente del disco a la memoria RAM. Esto permite entrenar con volúmenes de datos que superan la memoria física disponible, algo crítico en el uso de instancias con recursos limitados.

¹<https://huggingface.co/datasets/OpenAssistant/oasst1>

²https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered

³<https://huggingface.co/datasets/tatsu-lab/alpaca>

⁴<https://huggingface.co/datasets/stingning/ultrachat>

⁵<https://huggingface.co/datasets/bigcode/the-stack>

- **Almacenamiento Columnar:** A diferencia de formatos tradicionales como JSON o CSV, Arrow organiza la información por columnas. Esto optimiza el procesamiento de secuencias de texto al permitir lecturas parciales y una serialización extremadamente rápida.
- **Consistencia Estructural:** El uso de este formato garantiza que los campos *instruction*, *input* y *output* mantengan una tipificación estricta, facilitando la integración directa con los modelos de datos de Pydantic definidos en la aplicación.

6.1.3 Inyección de Conocimiento de Dominio (Fase Final)

Una vez que el modelo ha adquirido capacidades lingüísticas generales mediante el pre-entrenamiento, se procede a dotarlo de conocimiento especializado mediante el dataset sintético de Zurich, integrando dicho conocimiento directamente en los pesos de la red a través de una fase de fine-tuning supervisado.

6.1.4 Distribución del Dataset de Pre-entrenamiento

Para optimizar el rendimiento bajo restricciones de recursos, se ha definido la siguiente distribución de pesos para la fase inicial:

Categoría	Dataset	Peso	Propósito
Conversacional	OpenAssistant / Ultra-Chat	50%	Tono humano y consistencia en diálogos largos
Instrucciones	Alpaca	50%	Seguimiento de instrucciones y estructura resolutive

Table 4: Estrategia de mezcla de datos para el entrenamiento base del SLM (Fase Inicial).

6.2 Arquitectura del Modelo: Hybrid Transformer-GRU

La innovación técnica central de este modelo, denominado `tfm-slm`, radica en su arquitectura de bloques híbridos integrados. A diferencia de enfoques que simplemente concatenan capas, cada uno de los 12 bloques del modelo fusiona capacidades globales y secuenciales.

6.2.1 El Bloque Híbrido (HybridBlock)

Cada capa del modelo está compuesta por tres sub-componentes principales protegidos por conexiones residuales y normalización previa (*Pre-LayerNorm*):

1. **Multi-Head Attention (MHA):** Encargada de capturar dependencias globales y relaciones semánticas a larga distancia. Configurada con 12 cabezas de atención para un *hidden size* de 768.
2. **Gated Recurrent Unit (GRU):** Situada inmediatamente después de la atención, actúa como un filtro de refinamiento secuencial. Esta capa permite al modelo

mantener un estado interno más eficiente para patrones locales y estructurales, reduciendo la carga cognitiva del mecanismo de atención.

3. **Feed-Forward Network (MLP):** Una red de dos capas lineales con activación GELU y un factor de expansión de 4 (3072 neuronas), encargada del refinamiento de características de alto nivel.

6.2.2 Optimización del Cómputo de Atención: Flash Attention

El mecanismo de Multi-Head Attention estándar materializa la matriz $QK^\top \in \mathbb{R}^{L \times L}$ en memoria VRAM, lo que implica un coste de $O(L^2)$ en lecturas y escrituras HBM (*High Bandwidth Memory*). Para secuencias de longitud $L = 1024$ con 12 cabezas y *hidden size* 768, este coste de acceso a memoria domina el tiempo de cómputo real.

Para eliminar este cuello de botella, el modelo utiliza **Flash Attention v2.8.3** [9], implementada por el repositorio `dao-ailab/flash-attention`. Flash Attention reescribe el algoritmo de atención mediante tiling de bloques en SRAM, evitando materializar la matriz de atención completa en HBM. El resultado es matemáticamente idéntico al de la atención estándar pero con complejidad de acceso a memoria $O(L)$ en lugar de $O(L^2)$.

En la práctica, su integración en el entrenamiento del tfm-slm aporta dos beneficios concretos:

1. **Reducción de VRAM:** al no almacenar la matriz de atención completa, el pico de memoria GPU durante el forward pass disminuye significativamente, lo que permitió aumentar el batch físico de 4 a 16 sin superar los 24 GB de la NVIDIA L4.
2. **Aceleración del entrenamiento:** la reducción de transferencias HBM se traduce en un incremento de throughput de aproximadamente 2–4× respecto a la implementación estándar de PyTorch en hardware con arquitectura Ada Lovelace.

Flash Attention se compila durante la construcción de la imagen Docker, dado que requiere extensiones CUDA específicas para la GPU objetivo. Esta compilación está incluida en el `Dockerfile` del proyecto y se ejecuta automáticamente en la instancia EC2 como parte del pipeline de despliegue descrito en el Capítulo 9.

6.2.3 Especificaciones Técnicas y Parámetros

El modelo se ha dimensionado para ser competitivo en el rango de los Small Language Models, optimizando el ancho de banda de la arquitectura para hardware de 24 GB de VRAM. A continuación se detallan las especificaciones y una comparativa directa con el modelo de referencia en la industria, SmolLM2-135M.

Justificación del Tokenizer: Para esta arquitectura se ha seleccionado el tokenizer de **GPT-2** (50,257 tokens). Esta elección es estratégica por tres motivos:

1. **Eficiencia de Parámetros:** Al mantener un vocabulario moderado, se minimiza el *overhead* de memoria en la capa de *embeddings* (38M de parámetros), permitiendo dedicar una mayor fracción de la capacidad computacional a los bloques híbridos Transformer-GRU.

2. **Robustez Byte-level BPE:** El uso de codificación por pares de bytes (*Byte-Pair Encoding*) garantiza que el modelo pueda procesar cualquier secuencia de caracteres sin generar tokens desconocidos (*out-of-vocabulary*), algo crítico en la mezcla de datos técnicos y YAML.
3. **Estandarización:** Facilita la comparabilidad directa con otros modelos de la literatura técnica, asegurando que las métricas de rendimiento dependan de la arquitectura y no de variaciones en la tokenización.

Especificación	SmolLM2-135M	tfm-slm (Propuesto)
Arquitectura	Pure Transformer (Llama)	Hybrid Transformer-GRU
Parámetros Totales	135M	167M
Capas (Blocks)	30	12
Hidden Size (d_{model})	576	768
MLP Intermediate Size	1536	3072
Atención	GQA (9/3 heads)	MHA (12 heads)
Escalado de Memoria	Cuadrático $O(L^2)$	Híbrido (Linear-assisted)

Table 5: Comparativa arquitectónica: tfm-slm vs. SmolLM2-135M.

- **Capas:** 12 bloques híbridos. Aunque posee menos capas que SmolLM2, la densidad de información por bloque es superior gracias al componente GRU.
- **Dimensión del Modelo (d_{model}):** 768.
- **Contexto Máximo:** 1024 tokens.
- **Vocabulario:** 50,257 tokens (GPT-2 Tokenizer).
- **Weight Tying:** Se ha implementado el amarre de pesos entre la capa de *embeddings* y la cabeza de salida (*LM Head*), reduciendo el uso de memoria en un 18%.

6.3 Optimización y Estabilidad del Entrenamiento

Para garantizar que la arquitectura híbrida sea entrenable desde cero de forma estable, se han aplicado técnicas de inicialización y normalización avanzadas. Todos los pesos lineales se inicializan mediante una distribución normal con $\sigma = 0.02$, mientras que las capas de normalización comienzan con ganancia unidad y sesgo cero. El uso de *causal masking* en cada bloque asegura que la parte de atención no rompa la naturaleza autorregresiva necesaria para la generación de lenguaje, demostrando la eficiencia de la arquitectura híbrida en entornos de recursos limitados.

6.4 Expectativas de Convergencia y Métricas de Éxito

Para validar la viabilidad del entrenamiento se han definido métricas de éxito basadas en la literatura de los SLM y los resultados preliminares observados. La confianza en la funcionalidad final del modelo se sustenta en tres pilares:

1. **Calidad del Curado de Datos (Data Mix):** A diferencia del pre-entrenamiento masivo con datos web no filtrados, el uso de datasets de *instruction-tuning* como Alpaca y ShareGPT garantiza que el modelo aprenda a seguir instrucciones desde las fases iniciales. La inclusión de OpenAssistant aporta la naturalidad conversacional necesaria para un agente funcional.
2. **Leyes de Escala (Scaling Laws):** Con 167M de parámetros, el modelo se sitúa en un rango superior al GPT-2 Small (124M), el cual ya demostró capacidades emergentes de razonamiento gramatical. Los 43M de parámetros adicionales dedicados al componente GRU actúan como una reserva de memoria dedicada a la consistencia secuencial.
3. **Convergencia Observada:** El descenso de la función de pérdida (*loss*) hasta un valor de 1.72 en la primera época es un indicador sólido de estabilidad. Se estima que una programación de 5 épocas es suficiente para transicionar desde el aprendizaje de estructuras gramaticales básicas (Época 1) hacia la asociación de conceptos semánticos y el refinamiento del estilo de respuesta (Época 5).

Se define como éxito un modelo que demuestre coherencia sintáctica, adherencia al formato de diálogo *User/Assistant* y capacidad de recuperación de información básica sin degradación de la latencia en contextos largos, validando así la eficiencia de la arquitectura híbrida frente a los modelos de solo atención.

7 Evaluación y Benchmarking

La evaluación rigurosa de un modelo de lenguaje requiere ir más allá de la observación de la curva de pérdida durante el entrenamiento. Este capítulo documenta la evaluación cuantitativa del modelo híbrido tfm-slm mediante un benchmark exhaustivo de 10 métricas ejecutado sobre un conjunto de validación independiente, seguido de un análisis de los resultados obtenidos y su contexto respecto a modelos de referencia.

7.1 Entorno y Configuración del Benchmark

La evaluación se ejecuta en el mismo entorno de producción utilizado para el entrenamiento: una instancia **g6.2xlarge** de AWS equipada con una GPU NVIDIA L4 (24 GB VRAM). Esta decisión garantiza que las métricas de latencia y throughput sean representativas del rendimiento real en despliegue, y no de un entorno de laboratorio distinto al operacional.

El checkpoint evaluado corresponde al modelo final almacenado en S3 (**s3://tfm-slm-benchmarks/b** descargado automáticamente por el sistema de benchmarking. El conjunto de validación, igualmente descargado desde S3, consiste en **300.000 muestras** del dataset mixto de diálogos e instrucciones, procesadas en lotes de 64 tokens, lo que resulta en 4.688 iteraciones de evaluación.

Parámetro	Valor
Modelo evaluado	HybridBlock (12 capas, 768 oculto, 12 cabezas)
Dispositivo	CUDA (NVIDIA L4)
Muestras de validación	300.000
Tamaño de lote	64
Iteraciones totales	4.688
Tiempo total de evaluación	3.399,88 s (≈ 56 min)

Table 6: Configuración del entorno de benchmarking.

7.2 Métricas de Evaluación

El benchmark cubre diez métricas agrupadas en tres categorías: calidad del modelo, eficiencia computacional y huella de recursos.

7.2.1 Calidad del Modelo

Pérdida y Perplejidad. La **pérdida cross-entropy** final sobre el conjunto de validación es $\mathcal{L} = 1,091$. La **perplejidad**, definida como $PPL = e^{\mathcal{L}}$, alcanza un valor de **2,98**. La perplejidad puede interpretarse como el número efectivo de tokens igualmente probables que el modelo considera en cada posición: un valor de 2,98 indica que el modelo está altamente concentrado en sus predicciones, reduciendo la incertidumbre media a menos de 3 candidatos por token.

El log de evaluación muestra que la perplejidad se estabilizó en $\approx 2,98$ a partir del batch 300 (de 4.688), permaneciendo constante hasta el final. Esta estabilidad, visible

en la Figura ??, es indicativa de una distribución de validación uniforme y de un modelo bien generalizado sin sobreajuste localizado.

Precisión de Predicción. Se reportan tres niveles de precisión en la predicción del siguiente token:

Métrica	Valor	Interpretación
Top-1 Accuracy	79,75%	El token correcto es la primera predicción del modelo
Top-5 Accuracy	91,14%	El token correcto está entre las 5 primeras predicciones
Top-10 Accuracy	93,56%	El token correcto está entre las 10 primeras predicciones

Table 7: Métricas de precisión de predicción del modelo híbrido.

Una precisión Top-1 del 79,75% sobre 300.000 muestras es un resultado sólido para un modelo entrenado desde cero con 167M de parámetros. La diferencia de 11,39 puntos porcentuales entre Top-1 y Top-5 refleja la naturaleza inherentemente ambigua del lenguaje natural: en muchos contextos, múltiples tokens son igualmente válidos como continuación, y el modelo los sitúa correctamente entre sus candidatos principales.

7.2.2 Eficiencia Computacional

Latencia e Inferencia. La latencia media por token en modo de inferencia individual es de **7,00 ms**, equivalente a una cadencia de **142,76 tokens/segundo** en modo secuencial. Este valor es determinante para aplicaciones conversacionales, donde se percibe como fluido cualquier modelo que supere los 50 tokens/segundo.

En modo de inferencia por lotes (*batch inference*) se observan dos valores de throughput con naturalezas distintas. Durante la ejecución del benchmark, el log reporta de forma continua un rendimiento sostenido de **≈146.000 tokens/segundo** (con un arranque inicial de 142.170 tok/s en el primer bloque). Sin embargo, el resumen final generado por el sistema de benchmarking registra un valor de **90.268 tokens/segundo** bajo la clave `batch_throughput_tokens_per_sec`.

Esta discrepancia se explica por la diferencia de granularidad en la medición: el log refleja el throughput instantáneo por ventana de 10 batches, mientras que el JSON reporta el throughput efectivo global, que incluye el tiempo de carga del checkpoint desde S3, la inicialización del dataset y la descarga de artefactos al inicio de la sesión. El valor operativo representativo del rendimiento de la GPU en inferencia pura es el del log (**≈146.000 tok/s**), mientras que el del JSON refleja el coste real de un despliegue completo de principio a fin.

7.2.3 Huella de Recursos

Tamaño del Modelo. El modelo cuenta con **166.980.864 parámetros**, todos ellos entrenables, con un tamaño en disco de **636,98 MB**. Esto lo sitúa en la categoría de Small Language Model (SLM), equiparable en orden de magnitud a GPT-2 Medium (345M params) pero con una huella de memoria significativamente menor gracias a la arquitectura híbrida.

Memoria en Pico. El pico de memoria GPU durante la evaluación fue de **66,76 GB**. Este valor corresponde a la evaluación sobre los 300.000 samples con batch 64, e incluye las activaciones intermedias acumuladas para el cómputo de métricas. Durante la inferencia en producción (sin acumulación de gradientes ni activaciones), el consumo de memoria es considerablemente inferior y se mantiene dentro de los 24 GB disponibles en la L4.

7.3 Análisis de Estabilidad

Una de las conclusiones más relevantes del log de benchmarking es la notable estabilidad de todas las métricas a lo largo de las 4.688 iteraciones. La perplejidad oscila en un rango de apenas $\pm 0,01$ tras la fase inicial de estabilización (primeros 200 batches), y la precisión Top-1 se mantiene entre 79,7% y 79,8% durante el 95% de la evaluación.

Esta estabilidad no es trivial: indica que el modelo generaliza de forma homogénea a todos los subconjuntos del dataset de validación, sin degradación en muestras de dominios específicos ni varianza anómala que sugiera problemas de sobreajuste parcial.

Los picos de pérdida por batch observados en el log (valores ocasionales superiores a 1,2) son normales en evaluación de lenguaje natural y reflejan la presencia de muestras intrínsecamente difíciles (cambios abruptos de dominio, nombres propios infrecuentes, código), no un problema del modelo.

7.4 Resumen de Resultados

Métrica	Resultado	Categoría
Loss (cross-entropy)	1,091	Calidad
Perplejidad	2,98	Calidad
Top-1 Accuracy	79,75%	Calidad
Top-5 Accuracy	91,14%	Calidad
Top-10 Accuracy	93,56%	Calidad
Latencia por token	7,00 ms	Eficiencia
Throughput (single token)	142,76 tok/s	Eficiencia
Throughput (batch, GPU pura)	≈ 146.000 tok/s	Eficiencia
Throughput (batch, despliegue completo)	90.268 tok/s	Eficiencia
Parámetros totales	166.980.864	Recursos
Tamaño del modelo	636,98 MB	Recursos

Table 8: Resumen de las 10 métricas del benchmark del modelo híbrido tfm-slm.

Los resultados demuestran que la arquitectura híbrida Transformer-GRU alcanza un equilibrio eficiente entre calidad de predicción y coste computacional. Una perplejidad de 2,98 y una precisión Top-1 superior al 79% son resultados competitivos para un modelo de 167M de parámetros entrenado desde cero sobre datos públicos, validando la hipótesis central del trabajo: las arquitecturas híbridas pueden lograr rendimiento comparable al de los Transformers puros con una fracción de los requisitos de memoria.

8 Infraestructura Cloud y Análisis de Costes

Uno de los pilares de este trabajo es la demostración de la viabilidad económica del proyecto mediante el uso de infraestructura cloud optimizada y automatizada. La arquitectura de hardware seleccionada no solo responde a necesidades de rendimiento, sino que se integra en un flujo de despliegue continuo donde la intervención humana es mínima.

8.1 Ecosistema AWS: Provisión de Recursos

Para el despliegue del entorno de entrenamiento, se ha seleccionado Amazon Web Services (AWS) debido a su madurez en servicios de computación acelerada y su capacidad de ofrecer recursos de alto rendimiento bajo demanda.

8.1.1 Repositorio de Imágenes: Amazon ECR

Amazon Elastic Container Registry (ECR) actúa como el repositorio privado del proyecto. Se ha integrado en el flujo de CI/CD para almacenar las imágenes Docker del modelo tfm-slm, etiquetadas dinámicamente con el SHA del commit para garantizar la trazabilidad entre el código fuente y el binario desplegado.

8.2 Selección del Hardware: Familia G6 y NVIDIA L4

La elección de la potencia de cómputo es el factor determinante en el tiempo de entrenamiento de un Small Language Model. AWS ofrece diversas familias de instancias optimizadas para computación acelerada, cada una con un hardware GPU específico diseñado para distintos casos de uso.

8.2.1 Comparativa de Familias de Aceleración

Como se detalla en la Tabla 9, existen opciones que van desde la inferencia ligera (G4dn) hasta el entrenamiento masivo de LLMs (P5). Para este proyecto, se ha seleccionado la familia **G6** por su equilibrio óptimo entre capacidad de VRAM (24 GB) y coste operativo en modalidad Spot.

Instancia	Hardware GPU	Propósito Principal
G4dn	NVIDIA T4 (16 GB VRAM)	Inferencia y entrenamiento ligero
G5	NVIDIA A10G (24 GB VRAM)	Entrenamiento de modelos SLM / Gráficos
G6	NVIDIA L4 (24 GB VRAM)	Eficiencia energética y IA generativa
G6e	NVIDIA L40S (48 GB VRAM)	Fine-tuning de LLMs de tamaño medio
G7e	NVIDIA RTX PRO 6000 Server (96 GB VRAM)	Entrenamiento de LLMs y SLMs de alta capacidad
P4 / P5	NVIDIA A100 / H100	Cómputo de alto rendimiento (HPC)

Table 9: Ecosistema de aceleradores NVIDIA disponibles en AWS EC2.

8.2.2 Arquitectura NVIDIA Ada Lovelace en G6

La instancia seleccionada, **g6.2xlarge**, cuenta con un acelerador **NVIDIA L4**. Esta GPU de la familia Ada Lovelace destaca por sus núcleos Tensor de cuarta generación, soporte nativo de *bfloat16* y 24 GB de VRAM GDDR6, siendo especialmente eficiente energéticamente respecto a generaciones anteriores de VRAM equivalente como la A10G.

8.3 Gestión de Infraestructura mediante Terraform

Para garantizar la reproducibilidad del entorno, se ha implementado una arquitectura de **Infraestructura como Código (IaC)** utilizando Terraform. Toda la configuración reside en el directorio `app/terraform/`, lo que permite levantar el entorno completo de forma automática.

8.3.1 Automatización y Prerrequisitos

Gracias a la automatización implementada, el único requisito manual para el despliegue es la existencia de una clave SSH (*Key Pair*) denominada `tfm-slm` en la cuenta de AWS. El resto del proceso está codificado y es totalmente transparente para el desarrollador:

- **Selección de AMI:** Se utiliza la *Deep Learning Base AMI* (Figura 1) para garantizar un sistema operativo con drivers NVIDIA y CUDA preinstalados.
- **Provisionamiento Spot:** Terraform solicita la instancia buscando el menor coste en la región de España (`eu-south-2`).
- **Orquestación del Arranque:** Mediante scripts de *User Data*, la instancia se auto-configura al arrancar, instalando el motor Docker y lanzando el entrenamiento sin intervención manual.

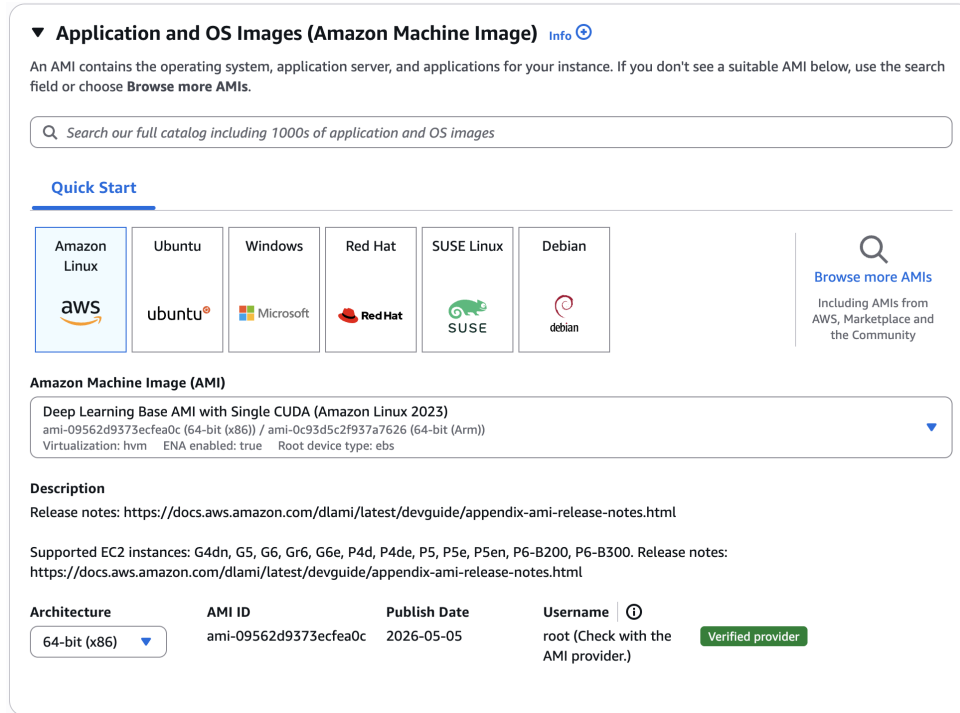


Figure 1: Selección automatizada de la AMI especializada para Deep Learning mediante Terraform.

8.4 Optimización de Memoria: Batching y Aprovechamiento de VRAM

Con el hardware desplegado, se procede a ajustar el software para aprovechar los 24 GB de la NVIDIA L4.

8.4.1 Estrategia de Gradient Accumulation

Se utiliza la técnica de acumulación para simular lotes grandes sin saturar la VRAM. La Tabla 10 resume la evolución desde el error inicial hasta la configuración óptima alcanzada durante las pruebas de estrés. La configuración conservadora (batch 4, acumulación 8) resultó estable pero lenta, mientras que la configuración optimizada (batch 16, acumulación 2) alcanzó un 97% de utilización de la GPU manteniendo la estabilidad del entrenamiento.

Configuración	Batch Físico	Acumulación	Batch Efectivo	Resultado
Base	32	1	32	Error OOM (Falla)
Conservadora	4	8	32	Estable (Lento)
Optimizada	16	2	32	Punto óptimo (97% Util.)

Table 10: Ajustes de memoria para maximizar el uso de la GPU NVIDIA L4.

8.5 Evaluación de Rendimiento vs Coste

La combinación de la arquitectura híbrida Transformer-GRU con la optimización de batching descrita en la sección anterior permitió alcanzar un 97% de utilización de la GPU NVIDIA L4 con un coste de instancia Spot inferior a 0,50 USD/hora en la región `eu-south-2`. Esta relación rendimiento/coste es la que valida la hipótesis central del trabajo: es posible entrenar un SLM competitivo sin infraestructura masiva.

8.6 Mejoras Iterativas de la Arquitectura durante el Entrenamiento

A lo largo del proceso de entrenamiento se identificaron y corrigieron limitaciones de la arquitectura inicial mediante un ciclo de desarrollo iterativo. Las mejoras se aplicaron de forma incremental, validando el impacto de cada cambio sobre la estabilidad del entrenamiento y las métricas de calidad.

8.6.1 Persistent GRU Hidden States

En la implementación inicial, el estado oculto del GRU se descartaba al final de cada bloque, haciendo que la capa recurrente se comportara de forma *stateless*. Se implementaron **estados ocultos persistentes** que fluyen a través de las 12 capas del modelo, manteniendo un tensor de estados compartido durante toda la secuencia de forward pass. Este cambio convirtió el GRU en un componente verdaderamente recurrente con memoria vertical entre bloques.

8.6.2 Posicionamiento Pre-Attention del GRU

Se modificó el orden de procesamiento dentro del bloque híbrido: el GRU ahora procesa la entrada **antes** que el mecanismo de atención. El nuevo orden ($\text{GRU} \rightarrow \text{MHA} \rightarrow \text{MLP}$) permite que la capa recurrente capture primero los patrones locales y secuenciales, entregando a la atención una representación ya refinada para la captura de dependencias globales. Esta inversión mejora la especialización de cada componente y reduce la redundancia entre ambos mecanismos.

8.6.3 Inicialización Ortogonal

Los pesos de entrada (`weight_ih`) y recurrentes (`weight_hh`) del GRU se inicializan mediante **inicialización ortogonal**, que garantiza que la matriz de pesos sea una isometría en el espacio vectorial. Esta técnica preserva la norma de los gradientes durante las primeras épocas de entrenamiento, reduciendo el riesgo de gradientes explosivos o en desaparición en la parte recurrente de la red.

8.6.4 Gradient Clipping

Se implementó *clipping* de gradientes con norma máxima `grad_clip_norm = 1.0`. El clipping se aplica después de cada *backward pass* y antes del paso del optimizador: si la norma L_2 del vector de gradientes supera el umbral, todos los gradientes se escalan proporcionalmente hacia abajo. Esta técnica es especialmente crítica en arquitecturas con

componentes recurrentes, donde los gradientes pueden acumularse de forma multiplicativa a lo largo de la secuencia.

8.6.5 Component Contribution Logging

Se desarrolló una herramienta de análisis (`analyzer.py`) que mide y reporta el **ratio de contribución** de cada componente del bloque híbrido (GRU, Attention y MLP) durante el entrenamiento. Esta herramienta proporciona visibilidad directa sobre si el modelo está aprendiendo a utilizar ambos mecanismos de forma balanceada o si uno domina al otro, permitiendo detectar degeneraciones tempranas antes de que afecten a las métricas de validación.

8.6.6 Validation Loop

Se añadió un bucle de validación periódico que evalúa el modelo sobre un subconjunto de datos no visto durante el entrenamiento. La ejecución del loop de validación se intercala con el entrenamiento cada N pasos, permitiendo detectar sobreajuste (*overfitting*) en tiempo real y aplicar *early stopping* si la perplejidad de validación deja de mejorar.

8.7 Evolución del Hardware: Migración a G7e

Las optimizaciones de software descritas en las secciones anteriores expresan al máximo los 24 GB de VRAM de la NVIDIA L4. Sin embargo, modelos de mayor capacidad o secuencias más largas requieren dar el siguiente paso en la jerarquía de hardware. La familia **G7e**, equipada con la **NVIDIA RTX PRO 6000 Server** (96 GB VRAM), representa la evolución natural para este proyecto: cuadruplica la memoria disponible manteniendo la misma arquitectura de despliegue basada en Terraform y Docker, sin cambios en el pipeline de entrenamiento. Esta migración está prevista como línea de trabajo futuro para explorar configuraciones de mayor escala (más capas, mayor *hidden size* o contextos de 4096 tokens) sin modificar la arquitectura híbrida base.

9 MLOps: Despliegue y Puesta en Producción

El ciclo de vida del modelo culmina con su despliegue en un entorno de producción real, sustentado por una arquitectura de software robusta y procesos de gobernanza de código altamente automatizados.

9.1 Arquitectura de Servicios e Idempotencia

La pipeline de datos se ha diseñado siguiendo los principios **SOLID**, transformando scripts procedimentales en servicios especializados y desacoplados como `DatasetDownloader` y `DatasetProcessor`. Un pilar fundamental es la **idempotencia**: cada servicio verifica el estado del sistema antes de ejecutar operaciones costosas, optimizando el uso de recursos en la nube.

9.2 Reproducibilidad y Gestión de Dependencias

Para garantizar que el modelo se ejecute en entornos idénticos (local, AWS, Kubernetes), se utiliza el gestor **uv** y su archivo de bloqueo `uv.lock`. Esto asegura el determinismo absoluto del entorno, optimiza la caché de Docker y proporciona seguridad mediante la verificación de *hashes* criptográficos.

9.3 Optimización del Pipeline de CI: Eficiencia y Velocidad

El flujo de integración continua en GitHub Actions se ha optimizado bajo dos vertientes: el tiempo de ejecución y el coste computacional.

9.3.1 Estrategia de Caché Global con Fallback

Dada la naturaleza pesada de las dependencias de Deep Learning, el sistema utiliza una caché compartida. GitHub Actions implementa una jerarquía donde el *runner* busca primero en la rama actual y luego realiza un *fallback* automático a `main`. Esto reduce el tiempo de build de 15 minutos a menos de 60 segundos.

9.3.2 Validación Basada en Revisión (PR-driven CI)

Se ha implementado una política de ejecución dirigida por **Pull Requests**. El pipeline de Docker solo se dispara cuando el código está listo para revisión, minimizando el consumo innecesario de recursos durante el desarrollo activo.

9.4 Integridad del Código y Gobernanza mediante Rulesets

Para asegurar la estabilidad de la rama principal (`main`), se ha configurado un **GitHub Repository Ruleset** que impone los siguientes controles de calidad:

- **Fusión exclusiva vía PR:** Se prohíben los cambios directos en la rama principal, obligando a un proceso de revisión formal.

- **Status Checks Obligatorios:** El pipeline de construcción de Docker (Docker Image CI) debe finalizar con éxito antes de permitir cualquier integración.
- **Revisión Asistida por IA:** Se ha integrado **GitHub Copilot Code Review** de forma automática en el ruleset. Esto garantiza que cada cambio sea analizado por modelos de lenguaje avanzados para detectar vulnerabilidades, errores lógicos y asegurar el cumplimiento de las mejores prácticas antes de la intervención humana.

9.5 Pruebas Automatizadas y Calidad de Código

La fiabilidad se garantiza mediante una suite de pruebas con **pytest**, utilizando técnicas de *mocking* para evitar dependencias externas. Estas pruebas se integran mediante *hooks* de *pre-commit*, manteniendo una cobertura superior al 95%.

Para asegurar la correcta monitorización del despliegue en producción, se han establecido protocolos de depuración en tiempo real. Debido a que el proceso de descarga e inicialización del contenedor de Deep Learning puede demorar entre 3 y 6 minutos debido al volumen de las imágenes, se utiliza el sistema **cloud-init** para seguir el progreso del aprovisionamiento:

```
1 sudo tail -f /var/log/cloud-init-output.log
```

Listing 4: Comando para monitorizar el aprovisionamiento inicial.

Una vez que el contenedor está activo, la trazabilidad del entrenamiento se mantiene mediante el acceso a los flujos de salida estándar de Docker, permitiendo una validación inmediata de la carga del modelo y la conexión con el sistema de persistencia en S3.

9.6 Infraestructura como Código con Terraform

La infraestructura de entrenamiento e inferencia se provisiona de forma reproducible mediante **Terraform** (versión del provider AWS ~¿5.0), con el estado remoto almacenado en un bucket S3 dedicado (**tfm-slm-terraform-state**, región **eu-south-2**) con cifrado en reposo y bloqueo de estado habilitados:

```
1 backend "s3" {
2   bucket      = "tfm-slm-terraform-state"
3   key         = "terraform.tfstate"
4   region      = "eu-south-2"
5   encrypt     = true
6   use_lockfile = true
7 }
```

Listing 5: Configuración del backend remoto de Terraform.

El módulo de infraestructura provisiona cuatro recursos principales:

1. **Instancia EC2 con GPU** (g7e.2xlarge por defecto): GPU NVIDIA RTX PRO Server 6000 (96 GB VRAM, arquitectura Ampere) con la AMI oficial de Amazon Deep Learning con drivers NVIDIA y PyTorch preinstalados. El *UserData* de la instancia automatiza el bootstrap completo: actualización del sistema, descarga del código desde S3, compilación de la imagen Docker con soporte CUDA (incluyendo **flash-attn**) y lanzamiento del modo de operación seleccionado.

2. **Buckets S3:** tres buckets con responsabilidades separadas — `tfm-slm-code` para el ZIP del código fuente, `tfm-slm-checkpoints` para los pesos del modelo (con versionado habilitado), y `tfm-slm-benchmarks` para los resultados de evaluación en formato JSON.
3. **Repositorio ECR (tfm-slm):** almacena la imagen Docker con `image_tag_mutability = MUTABLE` y política de ciclo de vida que retiene únicamente la última imagen, minimizando costes de almacenamiento.
4. **Roles IAM:** el perfil de instancia `tfm-slm-ec2-profile` otorga a la EC2 permisos de `EC2ContainerRegistryPowerUser` y `S3FullAccess`, eliminando la necesidad de credenciales estáticas en el contenedor.

9.6.1 Pipeline de Despliegue

El flujo de despliegue está orquestado por `deploy.py`, que actúa como punto de entrada único e interactivo. El operador selecciona el modo de operación (`train`, `inference` o `benchmark`), y el script ejecuta la siguiente secuencia de forma idempotente:

```

1 # 1. El operador lanza el script
2 python deploy.py          # selecciona modo (train/inference/benchmark)
3
4 # 2. deploy.py ejecuta automaticamente:
5 #   - archiva el código en tfm-slm-code.zip (sin __pycache__ ni ZIPs anteriores)
6 #   - sube el ZIP a s3://tfm-slm-code/
7 #   - verifica/crea el repositorio ECR
8 #   - verifica/crea el bucket de benchmarks
9
10 # 3. El operador aplica la infraestructura
11 cd app/terraform && terraform apply -auto-approve -lock=false
12
13 # 4. La EC2 arranca y ejecuta automaticamente build-and-train.sh
14 #   que construye la imagen Docker, la sube a ECR y lanza el modo elegido

```

Listing 6: Secuencia de despliegue desde la máquina local.

El script `build-and-train.sh` ejecutado en la EC2 unifica los tres modos bajo la misma lógica: construcción de la imagen, push a ECR y lanzamiento del contenedor con la variable de entorno `MODE` correspondiente. El `entrypoint.sh` del contenedor enruta la ejecución al comando `uv` adecuado según el modo:

```

1 MODE=${MODE:-train}
2 if [ "$MODE" = "inference" ]; then tfm-slm-inference
3 elif [ "$MODE" = "benchmark" ]; then tfm-slm-benchmark
4 else
5   tfm-slm
6 fi

```

Listing 7: Enrutamiento de modos en `entrypoint.sh`.

Esta arquitectura permite cambiar entre entrenamiento, inferencia y benchmarking con un único parámetro, sin modificar la imagen Docker ni la infraestructura subyacente.

9.6.2 Módulos Terraform: Configuración Detallada

El código Terraform se organiza en seis módulos (archivos `.tf`), cada uno con responsabilidad específica. El directorio raíz (`app/terraform/`) contiene estos módulos, con una estructura modular que permite fácil extensión y auditoría.

1. `provider.tf`: Configuración del Proveedor y Backend

Define el proveedor AWS, la versión requerida, y el backend remoto para estado:

```
1 terraform {
2   required_providers {
3     aws = {
4       source = "hashicorp/aws"
5       version = "~> 5.0"
6     }
7   }
8   backend "s3" {
9     bucket = "tfm-slm-terraform-state"
10    key     = "terraform.tfstate"
11    region  = "eu-south-2"
12    encrypt = true
13    use_lockfile = true
14  }
15 }
16
17 provider "aws" {
18   region = var.aws_region
19 }
```

Listing 8: provider.tf: configuración de proveedor y backend remoto

El backend S3 con `encrypt = true` proporciona protección en reposo. El `use_lockfile = true` previene aplicaciones simultáneas que causarían corrupción de estado. La versión `~>5.0` asegura compatibilidad con APIs modernas de AWS sin versiones obsoletas.

2. variables.tf: Parámetros Configurables

Define variables de entrada que parametrizan la infraestructura, permitiendo diferentes configuraciones sin modificar código:

```
1 variable "aws_region" {
2   default = "eu-south-2"
3 }
4
5 variable "instance_type" {
6   description = "EC2 instance type"
7   default     = "g7e.2xlarge"
8 }
9
10 variable "deployment_mode" {
11   description = "Deployment mode: train, inference, or benchmark"
12   validation {
13     condition = contains(["train", "inference", "benchmark"], var.deployment_mode)
14     error_message = "deployment_mode must be 'train', 'inference', or benchmark"
15   }
16 }
17
18 variable "spot_price" {
19   description = "Maximum price for Spot instance"
20   default     = "1.50"
21 }
```

Listing 9: variables.tf: variables clave

La variable `deployment_mode` incluye validación (bloque `validation`) que rechaza valores inválidos en tiempo de plan, evitando errores de runtime.

3. s3.tf: Buckets S3 y Versionado

Provisiona buckets S3 para código, checkpoints, datasets y benchmarks. Ejemplo (checkpoints):

```
1 resource "aws_s3_bucket" "checkpoints" {
2   bucket = "tfm-slm-checkpoints"
3   force_destroy = true
4   tags = {
5     Name = "tfm-slm-checkpoints"
```



```

6     Environment = "production"
7   }
8 }
9
10 resource "aws_s3_bucket_versioning" "checkpoints" {
11   bucket = aws_s3_bucket.checkpoints.id
12   versioning_configuration {
13     status = "Enabled"
14   }
15 }

```

Listing 10: s3.tf: bucket con versionado para checkpoints

El versionado habilitado permite recuperar checkpoints anteriores si un entrenamiento diverge. El `force_destroy = true` permite destruir el bucket (junto con contenido) via terraform destroy, útil para limpieza de desarrollo.

4. ec2.tf: Instancia EC2, Security Groups, IAM Roles, AMI

Es el módulo más complejo, provisioning la instancia GPU:

```

1 # Security Group: permite SSH entrada
2 resource "aws_security_group" "ec2" {
3   name           = "${var.project_name}-ec2-sg"
4   vpc_id         = data.aws_vpc.default.id
5
6   ingress {
7     from_port = 22
8     to_port   = 22
9     protocol  = "tcp"
10    cidr_blocks = ["0.0.0.0/0"]
11  }
12
13  egress {
14    from_port = 0
15    to_port   = 0
16    protocol  = "-1"
17    cidr_blocks = ["0.0.0.0/0"]
18  }
19 }
20
21 # IAM Role: permisos ECR + S3
22 resource "aws_iam_role" "ec2_role" {
23   name = "${var.project_name}-ec2-role"
24   assume_role_policy = jsonencode({
25     Version = "2012-10-17"
26     Statement = [{
27       Action = "sts:AssumeRole"
28       Effect = "Allow"
29       Principal = { Service = "ec2.amazonaws.com" }
30     }]
31   })
32 }
33
34 resource "aws_iam_role_policy_attachment" "ecr_power" {
35   role       = aws_iam_role.ec2_role.name
36   policy_arn = "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryPowerUser"
37 }
38
39 resource "aws_iam_role_policy_attachment" "s3_full" {
40   role       = aws_iam_role.ec2_role.name
41   policy_arn = "arn:aws:iam::aws:policy/AmazonS3FullAccess"
42 }

```

Listing 11: ec2.tf: seguridad y roles IAM (snippet)

La instancia EC2 obtiene AMI Deep Learning via data source (busca la más reciente de Amazon):

```

1 data "aws_ami" "dlami" {

```

```

2  most_recent = true
3  owners      = ["amazon"]
4  filter {
5    name     = "name"
6    values   = ["Deep Learning OSS Nvidia Driver AMI GPU PyTorch * (Amazon Linux 2023) *"]
7  }
8 }
9
10 resource "aws_instance" "training" {
11   ami              = data.aws_ami.dlami.id
12   instance_type    = var.instance_type
13   iam_instance_profile = aws_iam_instance_profile.ec2_profile.name
14
15   root_block_device {
16     volume_size = 100
17     volume_type = "gp3"
18   }
19
20   user_data = <<-EOF
21   #!/bin/bash
22   # Bootstrap: descarga codigo, construye Docker, inicia entrenamiento
23   dnf update -y
24   dnf install -y docker
25   systemctl start docker
26   aws s3 cp s3://tfm-slm-code/tfm-slm-code.zip .
27   unzip tfm-slm-code.zip
28   cd tfm-slm && docker build -t tfm-slm:latest .
29   # ... mas comandos de inicializacion
30 EOF
31 }

```

Listing 12: ec2.tf: selección automática de AMI y lanzamiento

El `user_data` es un script bash que se ejecuta automáticamente tras el boot de la instancia. Este script automatiza toda la secuencia: actualización del sistema, instalación de Docker, descarga de código desde S3, construcción de imagen, y lanzamiento del contenedor.

5. ecr.tf: Repositorio ECR y Política de Ciclo de Vida

Provisiona registro ECR con política de retención:

```

1 resource "aws_ecr_repository" "main" {
2   name              = var.project_name
3   image_tag_mutability = "Mutable"
4
5   image_scanning_configuration {
6     scan_on_push = true
7   }
8
9   force_delete = true
10 }
11
12 resource "aws_ecr_lifecycle_policy" "cleanup" {
13   repository = aws_ecr_repository.main.name
14
15   policy = jsonencode({
16     rules = [{
17       rulePriority = 1
18       description = "Keep only the last image to save space"
19       selection = {
20         tagStatus    = "any"
21         countType    = "imageCountMoreThan"
22         countNumber = 1
23       }
24       action = {
25         type = "expire"
26       }
27     }]
28   })

```

Listing 13: ecr.tf: repositorio ECR con garbage collection

La política de ciclo de vida retiene únicamente la última imagen, evitando acumulación que incrementaría costos de almacenamiento ECR. El escaneo de imágenes (`scan_on_push = true`) detecta vulnerabilidades de seguridad automáticamente en cada push.

6. outputs.tf: Valores de Salida

Exporta información útil para usuarios/scripts posteriores:

```

1 output "ec2_instance_id" {
2   value = aws_instance.training.id
3 }
4
5 output "ecr_repository_url" {
6   value = aws_ecr_repository.main.repository_url
7 }
8
9 output "s3_checkpoints_bucket" {
10  value = aws_s3_bucket.checkpoints.bucket
11 }
```

Listing 14: outputs.tf: valores exportados tras provisioning

Los outputs permiten que scripts externos (ej: `deploy.py`) obtengan IDs de recursos para operaciones post-provisioning sin re-descubrir via AWS CLI.

Flujo de Ejecución Terraform

El operador usa Terraform con esta secuencia:

```

1 # 1. Inicializar: descargar plugins, configurar backend
2 terraform init
3
4 # 2. Plan: visualizar que cambiara (sin aplicar)
5 terraform plan -var="deployment_mode=train"
6
7 # 3. Aplicar: provisionar recursos en AWS
8 terraform apply -auto-approve
9
10 # 4. Destruir: liberar recursos (limpieza)
11 terraform destroy -auto-approve
```

Listing 15: Operaciones Terraform típicas

El `-auto-approve` acepta cambios sin confirmación interactiva, útil para CI/CD. Sin él, Terraform espera confirmación manual.

9.6.3 Especificaciones de la Instancia EC2

La instancia `g7e.2xlarge` provisiona los siguientes recursos de compute:

G7e							
g7e.2xlarge	64.00	Intel Xeon Emerald Rapids	8	4	2	1 x NVIDIA RTX PRO Server 6000 GPU	96 GiB (1 x 96 GiB)

Figure 2: Especificaciones de instancia EC2 g7e.2xlarge: 8 vCPUs (Intel Xeon Emerald Rapids), 64 GB RAM, NVIDIA RTX PRO Server 6000 con 96 GB VRAM. Esta configuración proporciona suficiente memoria para entrenar modelos de 124M parámetros con batch sizes de 64 tokens sin spilling a disco.

9.6.4 Visualización de Componentes AWS

Los recursos de infraestructura se ilustran en las siguientes capturas, tomadas del panel de administración de AWS:

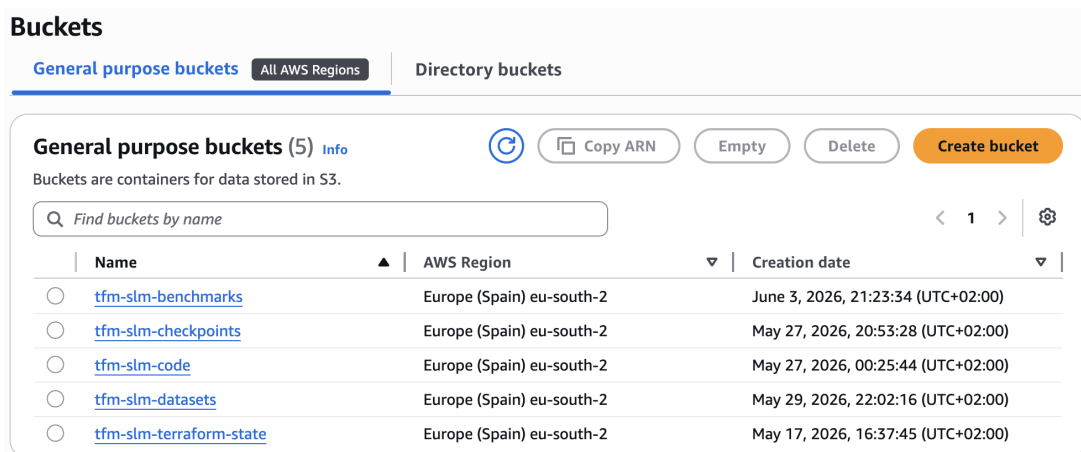


Figure 3: Vista general de buckets S3 en AWS. El proyecto utiliza múltiples buckets con responsabilidades separadas: código fuente, checkpoints de modelo, datasets procesados, estado de Terraform, y resultados de benchmarking. Esta separación permite auditoría granular, políticas de ciclo de vida independientes y recuperación de desastres eficiente.

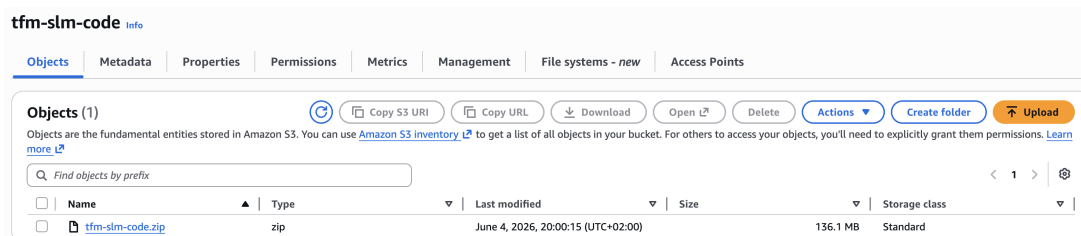


Figure 4: Bucket S3 `tfm-slm-code`: almacena el ZIP del código fuente de la aplicación. El `UserData` de la instancia EC2 descarga automáticamente esta fuente durante el bootstrap, permitiendo desacople entre gestión de infraestructura (Terraform) y gestión de código (Git/S3).

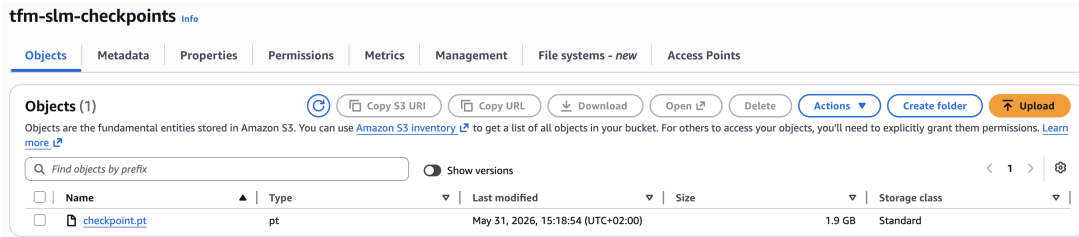


Figure 5: Bucket S3 **tfm-slm-checkpoints**: almacena los pesos entrenados del modelo con versionado habilitado. Después de cada época, el entrenamiento guarda un checkpoint aquí, permitiendo reanudación si hay interrupciones. El versionado S3 proporciona auditoría completa de cambios y capacidad de rollback.

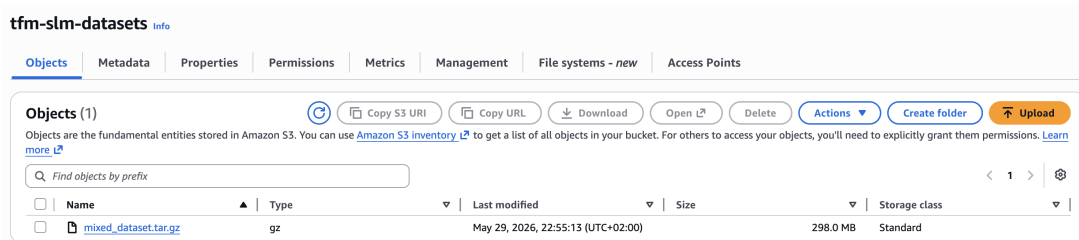


Figure 6: Bucket S3 **tfm-slm-datasets**: almacena los datasets preprocesados en formato Apache Arrow. Durante la inicialización de la EC2, estos archivos se descargan y se cachean localmente en el almacenamiento de instancia, minimizando lectura repetida desde S3 durante entrenamientos de múltiples épocas.

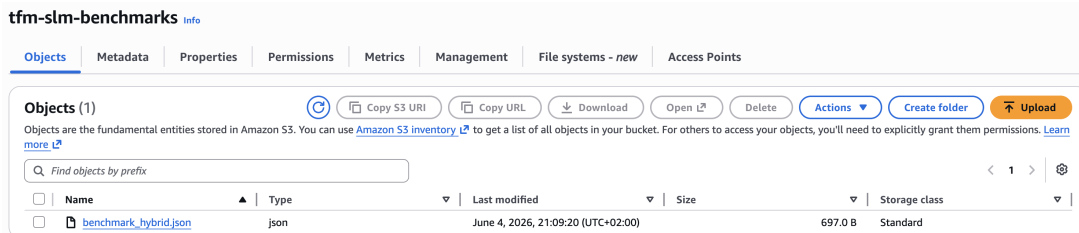


Figure 7: Bucket S3 **tfm-slm-benchmarks**: contiene resultados de evaluación y benchmarking en formato JSON. Después de cada entrenamiento, métricas de perplexity, velocidad de inferencia y consumo de VRAM se exportan aquí, permitiendo análisis histórico y comparativas entre ejecuciones.

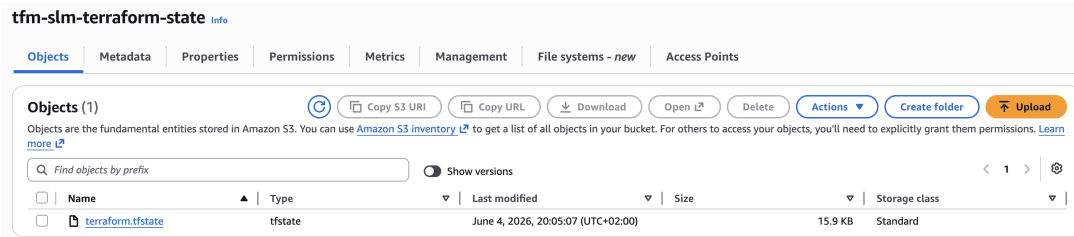


Figure 8: Bucket S3 **tfm-slm-terraform-state**: almacena el archivo de estado de Terraform (**terraform.tfstate**) con cifrado en reposo y versionado. Este bucket se trata como crítico; su pérdida requeriría reconstrucción manual de toda la infraestructura. El bloqueo de estado (mutex) previene aplicaciones simultáneas que causarían corrupción de estado.



Figure 9: Elastic Container Registry (ECR) **tfm-slm**: repositorio privado de imágenes Docker. El pipeline de CI publica nuevas imágenes aquí después de cada build exitoso en GitHub Actions. La política de ciclo de vida retiene únicamente la última imagen, evitando acumulación de imágenes obsoletas y controlando costes de almacenamiento.

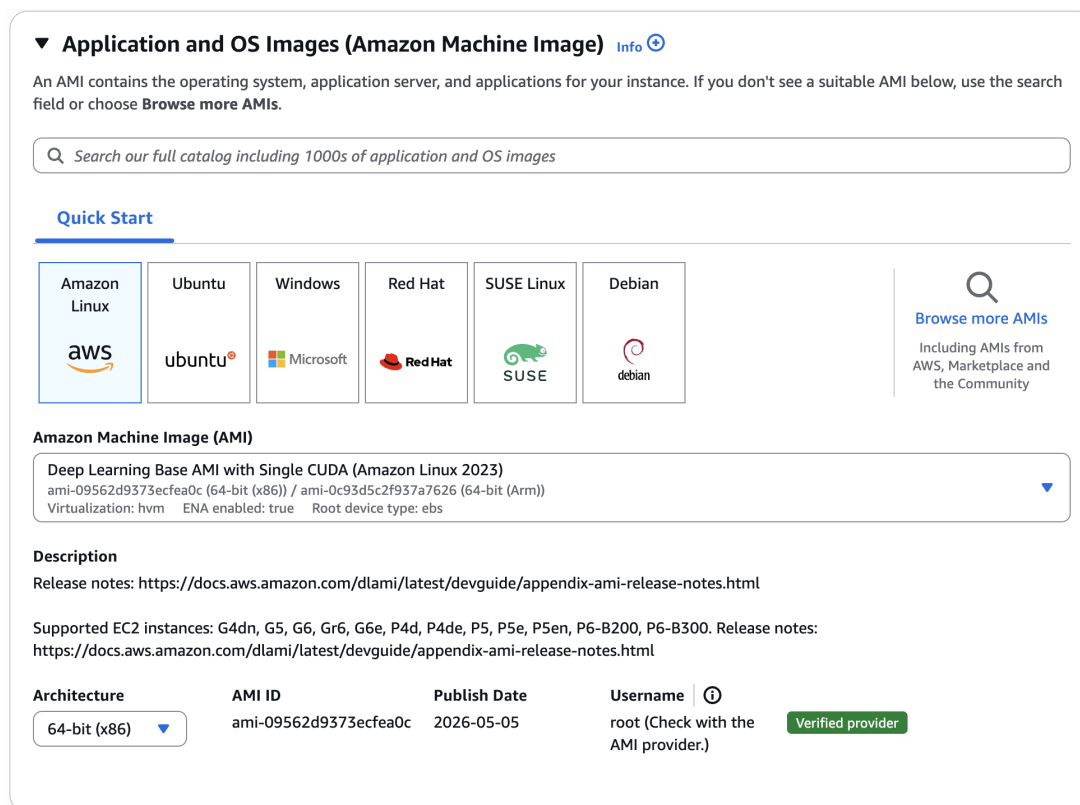


Figure 10: Amazon Machine Image (AMI) oficial de Deep Learning: preinstalada con drivers NVIDIA, CUDA, cuDNN y PyTorch. Usar esta AMI reduce el tiempo de bootstrap de la instancia EC2 de 20 minutos (compilación desde cero) a 3-6 minutos (solo descarga de contenedor y datasets). La AMI se especifica por ID en el módulo Terraform (`ami.tf`).

9.6.5 Topología y Flujo de Datos

La arquitectura de AWS forma una topología donde S3 actúa como hub central:

1. **Máquina Local** → **S3**: Desarrollador ejecuta `deploy.py`, que comprime código y lo sube a buckets S3 (`tfm-slm-code`, `tfm-slm-datasets`).
2. **Terraform**: Lee estado desde `tfm-slm-terraform-state`, provisiona EC2 (con AMI Deep Learning), ECR, y roles IAM.
3. **S3** → **EC2**: La instancia EC2 arranca y su script `UserData` descarga código y datasets desde S3.
4. **EC2** → **ECR**: En EC2, se construye imagen Docker (via `docker build`) y se sube a ECR como almacén central de imágenes.
5. **EC2** (Contenedor): El contenedor (ejecutándose localmente en EC2, o descargado desde ECR) entrena el modelo. Periódicamente guarda checkpoints en S3 y exporta métricas a `tfm-slm-benchmarks`.
6. **ECR**: Almacena imagen Docker construida como artifact reproducible para auditoría y futuros despliegues.

Resumido: **S3 es el hub de entrada/salida**. Código y datasets fluyen desde S3 hacia EC2; checkpoints y métricas fluyen desde EC2 hacia S3. ECR es almacén de imagen Docker, no participa en flujo de datos de entrenamiento.

9.7 Despliegue Local: Kubernetes con ArgoCD y Kustomize

Mientras que el entrenamiento y benchmarking se ejecutan en infraestructura AWS bajo control de Terraform, la distribución del modelo entrenado para consumo por usuarios finales se realiza mediante un clúster Kubernetes local orquestado por ArgoCD. Este despliegue local proporciona un entorno aislado y reproducible donde desarrolladores e investigadores pueden interactuar con el modelo mediante una interfaz web integrada.

9.7.1 Arquitectura del Clúster Local

El clúster local se provisiona usando **Kind** (Kubernetes in Docker), ejecutando completamente dentro de contenedores sin dependencias de máquinas virtuales. La configuración se gestiona mediante **Kustomize**, que proporciona capacidades de parametrización declarativa sin necesidad de template engines como Helm.

Los componentes principales del despliegue son:

1. **PersistentVolumeClaim** (`pvc.yaml`): Volumen local que almacena el checkpoint del modelo (~ 2 GB), permitiendo que los pods accedan al estado entrenado sin recargar desde red.
2. **ConfigMap** (`configmap.yaml`): Variables de entorno de la aplicación, incluyendo la ruta del checkpoint (`/app/data/checkpoint.pt`) y configuración de memoria para CUDA.
3. **Secrets** (`secrets.yaml`): Plantilla para credenciales de AWS (S3), permitiendo descargar modelos o logs desde la nube si es necesario.
4. **Chat Deployment** (`chat-deployment.yaml`): Deployment de Kubernetes que ejecuta la aplicación de inferencia, exponiendo un servidor web en puerto 8000 con interfaz de chat y API REST.
5. **Training Job** (`training-job.yaml`): Job de Kubernetes (opcional) que permite experimentar con entrenamiento en el clúster local utilizando GPU si está disponible.
6. **Service** (`service.yaml`): Servicio de Kubernetes que expone el Deployment, permitiendo acceso interno dentro del clúster y port-forwarding hacia localhost.

9.7.2 Flujo GitOps con ArgoCD

ArgoCD actúa como el **controlador de sincronización** entre el estado deseado (almacenado en Git bajo `argocd-cluster/manifests/`) y el estado actual del clúster. El manifiesto de aplicación ArgoCD (`application.yaml`) especifica:


```

1 spec:
2   source:
3     repoURL: 'git://host.docker.internal/tfm-slm'
4     path: argocd-cluster/manifests
5   syncPolicy:
6     automated:
7       prune: true
8       selfHeal: true
9     syncOptions:
10      - CreateNamespace=true

```

Listing 16: Configuración de sincronización GitOps en ArgoCD.

Las opciones de sincronización proporcionan:

- **prune: true:** Si un recurso es eliminado de Git, ArgoCD lo elimina automáticamente del clúster.
- **selfHeal: true:** Si un pod muere o es modificado manualmente, ArgoCD lo restaura al estado deseado.
- **CreateNamespace: true:** Crea automáticamente el namespace `tfm-slm` si no existe.

9.7.3 Ciclo de Despliegue Típico

El flujo operacional para desplegar una nueva versión del modelo es:

1. **Cambios en Git:** El usuario modifica los manifests en `argocd-cluster/manifests/` (ej: actualizar etiqueta de imagen, cambiar configuración de memoria).
2. **Commit y Push:** Los cambios se commiterizan en el repositorio Git local (simulado con Git daemon en la máquina del desarrollador).
3. **Detección de ArgoCD:** ArgoCD monitorea continuamente el repositorio y detecta divergencia entre Git y el clúster en menos de 3 segundos (configurable).
4. **Sincronización Automática:** Sin intervención manual, ArgoCD aplica los cambios mediante `kubectl apply` automático, realizando *rolling updates* de los pods sin downtime.
5. **Rollback Inmediato:** Si hay problemas, la reversión se realiza simplemente haciendo `revert` del commit en Git; ArgoCD sincroniza nuevamente hacia el estado anterior.

9.7.4 Carga de Checkpoints y Uso Local

Dado que el clúster Kind corre aislado dentro de Docker, el checkpoint debe transferirse explícitamente:

```

1 # Obtener nombre del pod de chat
2 POD_NAME=$(kubectl get pods -n tfm-slm -l app=tfm-slm-chat \
3   -o jsonpath='{.items[0].metadata.name}')
4
5 # Copiar checkpoint desde el Mac hacia el volumen persistente del pod
6 kubectl cp .output/checkpoint.pt tfm-slm/$POD_NAME:/app/data/checkpoint.pt
7

```

```
8 # Reiniciar el pod para que cargue el checkpoint
9 kubectl rollout restart deployment/tfm-slm-chat -n tfm-slm
```

Listing 17: Transferencia de checkpoint al clúster local.

Una vez reiniciado, el modelo está disponible para inferencia. Los usuarios pueden acceder mediante:

1. **Interfaz Web:** Navegador en `http://localhost:8000`, proporcionando chat visual en tiempo real.
2. **API REST:** Consultas directas con `curl` o clientes HTTP, permitiendo integración en aplicaciones externas.

9.7.5 Ventajas del Despliegue Local

Este enfoque ofrece varias ventajas sobre un despliegue directo en la máquina local:

- **Aislamiento:** El modelo corre en su propio namespace, sin contaminar dependencias del sistema operativo.
- **Escalabilidad:** Pasar de 1 a múltiples réplicas es trivial: cambiar `spec.replicas` en el manifest.
- **Reproducibilidad:** Cualquier desarrollador puede clonar el repositorio y desplegar la misma configuración.
- **Auditoría:** Toda decisión de despliegue queda registrada en el historial de Git.
- **Zero Downtime:** Las actualizaciones de imagen ocurren sin interrumpir el servicio gracias a rolling updates.

10 Conclusiones y Trabajo Futuro

Este capítulo resume los hallazgos del proyecto y las posibles líneas de investigación.

10.1 Viabilidad de los SLM Híbridos

Conclusiones sobre si un modelo de estas características es suficiente para casos de uso empresariales específicos.

10.2 Lecciones Aprendidas

Reflexión sobre los desafíos de ingeniería encontrados durante el desarrollo CUDA y el despliegue MLOps.

10.3 Trabajo Futuro

Posibilidades de mejora: cuantización del modelo, soporte para arquitecturas de inferencia como vLLM u Ollama, y fine-tuning con datos de dominio específico.

11 Anexo: Guía Rápida de Comandos

12 Anexo: Configuración del Backend

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in neural information processing systems*, pp. 5998–6008, 2017.
- [2] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [3] S. Gunasekar, Y. Zhang, J. Aneja, C. C. T. Mendes, A. Del Giorno, S. Gopi, M. Javaheripi, P. Kauffmann, G. de Rosa, O. Saarikivi, *et al.*, “Textbooks are all you need,” *arXiv preprint arXiv:2306.11644*, 2023.
- [4] L. B. Allal, A. Lozhkov, E. Bakouch, L. von Werra, and T. Wolf, “Smollm - blazingly fast and remarkably powerful.” <https://huggingface.co/blog/smollm>, 2024. Accessed: 2026-05-03.
- [5] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using rnn encoder-decoder for statistical machine translation,” *arXiv preprint arXiv:1406.1078*, 2014.
- [6] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv preprint arXiv:2312.00752*, 2023.
- [7] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [8] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, “Empirical evaluation of gated recurrent neural networks on sequence modeling,” *arXiv preprint arXiv:1412.3555*, 2014. Presented at the NIPS 2014 Deep Learning and Representation Learning Workshop.
- [9] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems*, vol. 35, pp. 16344–16359, 2022. Version 2.8.3 utilizada en este trabajo: <https://github.com/dao-ailab/flash-attention>.